

TaskSphere **(Smart Task Management System)**

A Project Report

Submitted in partial fulfilment of the requirements for the award of the
Degree
Of

BACHELOR OF TECHNOLOGY IN COMPUTER SCIENCE & ENGINEERING

Raja Kumar Rajak
(001CSE22GT019)

Sunil Kumar
(001CSE22GT024)



UNDER THE SUPERVISION OF

Supervisor -

Mr. Abhishek Kumar Singh
(Assistant Professor)

DEPT. OF COMPUTER SCIENCE & ENGINEERING

HOD -

MRS. KIRTI VERMA
(Assistant Professor)

DEPT. OF COMPUTER SCIENCE & ENGINEERING

RAM KRISHNA DHARMARTH FOUNDATION (RKDF)
UNIVERSITY, RANCHI
(PUNDAG, RANCHI, JHARKHAND-834004)



June, 2026

RKDF UNIVERSITY RANCHI
DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING
(PUNDAG, RANCHI, JHARKHAND-834004)

CANDIDATE'S DECLARATION

We hereby certify that the dissertation entitled “**TaskSphere**” which is being submitted in the partial fulfilment of the requirement for the award of Bachelor of Technology in **Computer Science & Engineering**, is a record of our own work carried out under the supervision and guidance of **Mr. Abhishek Kumar Singh** Assistant Professor, Department of Computer Science & Engineering, and **Mrs. Kirti Verma**, HOD, Department of Computer Science & Engineering, RKDF University Ranchi.

All information in this document has been obtained and obtained and presented as per academic rules and rules and ethical conduct.

To the best of my knowledge, the contents presented in this dissertation have not been submitted elsewhere for the award of any other degree/diploma.

Date :

Raja Kumar Rajak
(001CSE22GT019)

Place: Ranchi

Sunil Kumar
(001CSE22GT024)



RKDF UNIVERSITY RANCHI
DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING
(PUNDAG, RANCHI, JHARKHAND-834004)

CERTIFICATE OF APPROVAL

This is to certify that the work embodied in this Project Report entitled “**TaskSphere** ”, is carried out by **Raja Kumar Rajak (001CSE22GT019)** , **Sunil Kumar (001CSE22GT024)** has been approved for the degree of **Bachelor of Technology in Computer Science & Engineering** , Ram Krishna Dharmarth Foundation (RKDF) University, Ranchi.

Date:

Place: Ranchi

Internal Examiner

External Examiner

Head of Department



RKDF UNIVERSITY, RANCHI

ESTABLISHED UNDER GOVT. OF JHARKHAND ACT & REGISTERED UNDER 2(f) 1956

Ref. No.: RKDF/RANCHI/2026/CSE-PRJ08

Date: 12/05 /2026

TO WHOM IT MAY CONCERN

Subject: Approval of Project Synopsis

This is to certify that the project synopses submitted by RAJA KUMAR RAJAK, SUNIL KUMAR of B.Tech CSE VIII session - 2022-2026, have been reviewed and approved by the Department of Computer Science & Engineering

Name: RAJA KUMAR RAJAK
Enrolment No.: 001CSE22GT019

Name: SUNIL KUMAR
Enrolment No.: 001CSE22GT024

Project Title: TaskSphere
Supervisor: Mr Abhishek Kumar Singh

The submitted synopses meet the academic standards and research guidelines set by the department, and the students are hereby granted approval to proceed with their respective projects under the guidance of their assigned supervisors and any change in the topic must be done after approval of department.

We wish them success in their research and development efforts.

Kirti Verma
HOD,
Department of Computer Science & Engineering,
RKDF University, Ranchi , Jharkhand

Address: Argora-Kathal More Road, Dhipatoli-Pundag, Opp.- Water Tank Ranchi-834004
Email : info@rkdfbranchi.in Website : www.rkdfuniversity.org



RKDF UNIVERSITY RANCHI
DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING
(PUNDAG, RANCHI, JHARKHAND-834004)

**CERTIFICATE OF DECLARATION FROM THE
SUPERVISOR**

This is to certify that the work presented in the dissertation entitled “**TaskSphere**” in partial fulfilment of the requirement for the award of Degree of (**B. Tech (Computer Science & Engineering)**) at **RKDF UNIVERSITY RANCHI, JHARKHAND** is an authentic work carried out by **Raja Kumar Rajak** (001CSE22GT019) , **Sunil Kumar** (001CSE22GT024) under my supervision and guidance.

To the best of my knowledge, the content of this thesis does not form a basis for the award of any previous Degree to anyone else.

Date:

SUPERVISOR:-
Mr. Abhishek Kumar Singh
Assistant Professor
Department of Computer Science &
Engineering
RKDF University, Ranchi

HOD:-
Mrs. Kirti Verma
Assistant Professor
Department of Computer Science &
Engineering
RKDF University, Ranchi

ACKNOWLEDGEMENT

Our deepest thanks and admiration go out to my advisor, Mr. Abhishek Kumar Singh, an assistant professor in the computer science engineering department of RKDF University, Pundag in Ranchi, Jharkhand for his insightful criticism, insightful recommendations, and encouraging words, which infused renewed zeal into every choice made throughout the course of this project. We will always be appreciative of his politeness and his assistance with my studies.

In particular, We would like to thank Prof. (Dr.) S Chatterjee, Vice Chancellor, Dr. Amit Kumar Pandey, Registrar, RKDF University, Ranchi, Jharkhand, and Mrs Kirti Verma, Assistant Professor of Computer Science & Engineering, for their helpful comments and recommendations throughout the course of my project.

We also want to thank the professors and other employees of the Computer Science & Engineering Department at RKDF University, Ranchi who helped me with my research in some way.

A special gratitude goes out to our parents for their support and efforts, as well as to our friends for helping us carry out our experimental work properly. They gave me the necessary support and encouraging words We needed to achieve.

Raja Kumar Rajak
(001CSE22GT019)
B. TECH (C.S.E)
2022-2026

Sunil Kumar
(001CSE22GT024)
B. TECH (C.S.E)
2022-2026

ABSTRACT

TaskSphere is a web-based academic project and task management system developed specifically for universities and engineering colleges to simplify project coordination, task allocation, and progress tracking. Traditional academic project management often suffers from poor communication, lack of accountability, and inefficient manual tracking methods. TaskSphere addresses these issues by providing a centralized platform for managing academic teams and project activities.

The system supports three types of users: Super Admin, Teacher, and Student. The Super Admin manages users and system operations, Teachers create project teams and assign tasks with deadlines, while Students update task progress through a structured workflow consisting of Pending, In Progress, and Completed states. Team leaders can upload proof of task completion, which is securely stored and linked to project records. The system also maintains activity logs to track important actions and improve transparency among teachers and students.

TaskSphere is developed using Java 21 and Spring Boot 3.3.6. The backend uses Spring JdbcTemplate with raw SQL queries for database operations, while MySQL 8 is used as the relational database. The frontend is built using Thymeleaf, HTML, CSS, and JavaScript. Security features include session-based authentication, BCrypt password hashing, role-based access control, and secure password reset functionality using UUID tokens and email verification. Automated email reminders are sent to teams before task deadlines using scheduled background jobs.

The project demonstrates how a domain-specific academic management platform can improve collaboration, accountability, and task monitoring more effectively than generic project management tools while remaining lightweight, secure, and deployable within existing university infrastructure.

Keywords: Spring Boot, Java 21, JdbcTemplate, Thymeleaf, MySQL 8, Session-Based Authentication, BCrypt, Role-Based Access Control, Academic Project Management, Email Notification, Activity Log.

TABLE OF CONTENTS

<u>Contents</u>	<u>Page No</u>
Candidate's Declaration.....	2
Certificate of Approval.....	3
Supervisor Declaration	4
Acknowledgement.....	5
Abstract	6
Chapter 1: Introduction	9
1.1 Overview of the Project Domain.....	9
1.2 Problem Statement	10
1.3 Objectives of the Project	10
1.4 Scope of the Project.....	11
1.5 Methodology Overview.....	12
Chapter 2: Literature Review / Existing System	13
2.1 Overview of Similar Systems.....	13
2.2 Limitations of Existing Solutions.....	15
2.3 Research Gap.....	15
Chapter 3: System Requirements	17
3.1 Hardware Requirements	17
3.2 Software Requirements	17
3.3 Functional Requirements.....	18
3.4 Non-Functional Requirements.....	19
Chapter 4: System Design	21
4.1 System Architecture	21
4.2 Data Flow Diagrams.....	23
4.3 Use Case Diagram.....	26
4.4 ER Diagram and Database Schema.....	27
4.5 Database Tables.....	29
4.6 Task Workflow Flowchart.....	32
Chapter 5: Implementation	34
5.1 Project Structure	34
5.2 Technology Stack.....	35
5.3 Module-wise Implementation.....	36
5.4 Key Code Snippets	42
5.5 Interface Screens	45

Chapter 6: Testing & Validation	54
6.1 Testing Approach	54
6.2 Authentication Testing	54
6.3 Teacher Module Testing.....	56
6.4 Student Module Testing	58
6.5 Database Validation	59
6.6 Browser Compatibility Testing	60
Chapter 7: Results & Discussion.....	61
7.1 Functional Outcomes.....	61
7.2 Advantages of the Proposed System	62
7.3 Comparison with Existing Solutions	62
Chapter 8: Conclusion & Future Scope.....	64
8.1 Summary of Achievements	64
8.2 Limitations of the Current System	65
8.3 Future Enhancements	65
References / Bibliography	67
Appendix A: Source Code.....	70
Appendix B: Installation Guide.....	72

CHAPTER 1: INTRODUCTION

1.1 Overview of the Project Domain

Managing academic project assignments is one of the recurring administrative issues among engineering colleges and universities. In a standard semester, one teacher can help several project groups in parallel that are different frequencies (complete or unfinished), have different deadlines, and are often communicated (if at all) through tools that are not synchronized, or by message applications or by their own personal email threads or other unstructured meetings. On the other hand, students often do not have a single, clear, well organized picture of obligation, task timeline, and the product of work that is expected from them when completed.

This lack of structure creates actual academic danger in addition to just the hassle for participants. Students forget to submit if there is no formal system in place that makes the deadlines. Students forget to submit if there is no official system that sets the submission deadline. If there's no evidence at task completion, the evidence is completed and it is unverifiable. If no audit trail is available, then it will not be possible to come to any resolution about timelines and contributions. Errors are prone to occur when administrative staff have to enter users in the system by hand on spreadsheets and records begin to become out of synch.

TaskSphere's purpose was to address just such issues. It's a web-based academic tasks management system specifically created for the college environment, where the duties and authority of admins, teachers and students are well defined and have a predefined chain of command. A hierarchy is directly replicated in software with three user roles (Super Admin, Teacher, Student) and a separate interface along with a limited number of actions that match the real-life academic responsibilities.

The project has been built from scratch as a final year B.Tech project, explicitly with a purpose of creating deployable system that University department could use without having to perform cloud subscription, costly licensing calculation, substantial IT infrastructure investment.

In many educational institution, communication tools like email threads, spreadsheets, and messaging systems continue to be used for academic project management. In most of the education streams, communication remains divided with the help of solutions like

messaging applications, spreadsheet, and email streams for academic project coordination. They do not have any features of centralized monitoring, systematic workflows, or accountability, and this results in poor project supervision and high risk of errors.

1.2 Problem Statement

The following specific deficiencies were found by observing the current academic project management practices, and serve as the main motivation for the development of TaskSphere :

- There is no centralized system to use for allocating project tasks to teams of students, and hence tasks are communicated in an informal manner which often leads to misunderstandings.
- All progress tracking is dependent upon the student's self-report and there is no way for teachers to independently verify progress.
- They don't have a structured task status workflow so one can claim to complete the task without passing through the earlier ones.
- Reminders of deadlines aren't present or involve manual effort from faculty to communicate, which can result in deadlines being missed.
- There is no audit trail generating who did what at what time to enforce accountability.
- The manual process for user account management (addition or deletion of students, teachers and staff) is open to inconsistencies and loss of data.
- Out-of-band transmission of sensitive academic information, such as task assignments, deadlines, evidence of task completion.
- Teachers cannot centrally oversee, approve and for submission of completed academic work with proper documentation.

TaskSphere devises a structured academic task management system with a series of special features to solve the above problems.

1.3 Objectives of the Project

At the beginning of the work project, the following goals were set and all design and implementation decisions made accordingly:

1. To create user role specific web-portal for three different roles – Super Admin, Teacher and Student along with role specific dashboards and access control for each HTTP request with no exception.
2. To give the Super Admin uniform management capabilities for teacher account creation, editing, and deletion and have all teacher and student passwords stored as BCrypt hashes.
3. To empower teachers to create project teams, delegate tasks to team members with deadlines, manage different team members and designate team leaders, monitor project status and more from one dashboard.
4. To ensure that a task workflow (Pending → In Progress → Completed) can not be changed to an invalid state at the service layer, considering no invalid changes to be possible in the database.
5. To send automated email notices to everyone on the student team a fixed number of days in advance of task's due date using a Spring `@Scheduled` daily cron job.
6. To keep the activity log of all relevant actions done by an actor with their identity, type of action, description and time stamp.
7. Support teacher workflow to approve, reject and hold for controlled re- upload of submitted files.
8. To give a single place to be notified about deadlines, submission reviews, and team notification.

1.4 Scope of the Project

At present (version v1.0) the scope of the TaskSphere is defined as follows:

In scope: Three-role user management (Super Admin, Teacher, Student); Team creation and team lifecycle management; Task assignment and task tracking to deadlines; Forced 3-state user's lifecycle in teams; Forced picture upload for finishing user's tasks; Teacher's review and approval of users' tasks; Forced re-upload of user's help requests; Centralized notification panel; Activity tracking dashboard; Real-time picture validation in forms, thanks to AJAX; Forced picture upload at login (for teachers and students); SMTP-based deadlines reminders, thanks to the Cron job; Complete activity tracking; Token-based password reset via e-mail; Forced password reset at first use (for teachers and students); Team form AJAX iframe real-time user identifier validation.

Historic processes: Real time in browser messaging/chatting; integration with video conferencing services; grade management / submission of grades; external Learning Management Systems (LMS) integration; multi-department/multi-tenant; mobile app. These will be recognized as future improvements in Chapter 8.

1.5 Methodology Overview

TaskSphere was developed in an iterative (Sprint) fashion that adheres to Agile principles. The following five 2-week sprints were designed, and implemented in succession:

1. Sprint 1 – Foundation: Initializing the Database schema, spring-boot project, configuring Maven dependencies, session based authentication, role enumeration, and HomeController routing.
2. Sprint 2 – Admin Module: Super Admin dashboard, Teacher and Student CRUD operations using PortalService, BCrypt password hashing, first login enforcement.
3. Sprint 3 – Teacher Module: Team creation, dynamic team member forms, TeamService business logic, activity log integration, AJAX identifier validation endpoints, and team detail and edit pages.
4. Sprint 4 – Student Module: Student dashboard, Task status update with TeamStatus transition enforcement, FileStorageService for uploading screenshots, WebConfig resource handler for uploading files.
5. Sprint 5 – Notifications, Email & Hardening: DeadlineReminderScheduler cron job, EmailService SMTP integration, PasswordResetService token workflow, UI polish, review approval workflow, notification system enhancement, dashboard usability improvements, and manual end-to-end validation testing.

CHAPTER 2: LITERATURE REVIEW / EXISTING SYSTEM

2.1 Overview of Similar Systems

2.1.1 Trello

Created by Atlassian and released in 2011, Trello is a popular kanban-board app for displaying tasks as Trello cards on a visual board. It enables creation of lists, card assignment to users, setting due dates and adding labels, attachments etc to cards. Trello is developed with the basic goal is basically as a general device, which is why it has become popular throughout lots of industries. It however, does not recognize any kind of user hierarchy specific to the University. There is no native Super Admin role to be able to manage any teacher's account above, there is no enrollment number field for identifying students, there is no set status workflow (cards can be dragged anywhere and in any order), there is no enforced evidence-upload requirement for task completion and no built in automated email reminder system that is scoped to academic deadlines. Additionally, its free feature constricts how much you can do with its power-ups by preventing even some workarounds to these restrictions without paying for the game.

2.1.2 Jira

Jira is by Atlassian, another professional project management and bug catching device made to primarily supply the software program growth staff. Supplies strong workflow customisation, counting sprint arranging, Gantt style roadmaps, and extensive reporting. But there's a lot of complexity to Jira's power. A significant amount of time must be spent when setting up a project, creating workflows, configuring user roles and understanding the interface, which isn't really feasible for the faculty who are experts on their academic research but not on project management software. Another aspect of Jira that is too costly to be viable for a single university department is the number of licensing fees that must be paid for teams of more than ten users.

2.1.3 Google Classroom

Google Classroom was launched by Google in 2014 as a service for educational purposes, to create and distribute assignments, grade and gather student work. It is suitable in

individual assignment management and simply easy to use to the many students. It is however not natively supporting team based project tracking with the use of concrete project states for workflow. It does not have any notion upon project team and its leader making a single progress on a single task status. There is no team level deadline reminder configured, no activity audit log system and the very last thing that has to be uploaded or attached to proof of work completed is a screenshot. Furthermore, Google Classroom relies on the entire Google Workspace ecosystem, and everyone has to use a Google account related to their institutions, making it another cloud dependency.

2.1.4 Microsoft Teams with Planner

Using Microsoft Teams along with Microsoft Planner, users can receive notifications as well as use task management alongside the overall communication platform. Planner: No transition enforcement in progress state= etc. (Not Started, In Progress or Completed states are simple categories where any user may move any task to any state at any time). No evidence of completion is required, no automatic email reminder scheduler, no activity log, and no faculty, administration or Dean hierarchy similar to the university. Its primary product is communication and the project management features are secondary and a by-product.

2.1.5 Asana

Asana is a work management system with project durations, dampen relationships, and staff collaboration abilities. It can be shared using custom workflows and rules for automation, but there are limits to how much they can be used within the context of meaningful use, which for these features depend on a Business or Enterprise subscription. Academic hierarchies are not modelled on the platform natively, meaning there is no difference between head administrator, faculty member and student. Requirements such as evidence-upload on task completion, tracking enrollment number, automatic reminder e-mail for tasks based on team deadlines are not provided out of the box.

2.1.6 Spreadsheet-Based Academic Tracking

Most schools still actively operate academic project tasks on spreadsheets, with using messaging applications and informal communication protocols. Enables basic task listing

and recording of deadlines using spreadsheets, but does not include central workflow enforcement, role-based access control, automated reminders, audit logging, evidence-based task verification. Handwritten and paper-based tracking of spreadsheets also brings a risk of misupdates, duplicate entries, and missed deadlines, as well as notifying one faculty member of the other. With these restrictions, it becomes evident that a specific academic workflow management platform, like TaskSphere, is needed

2.2 Feature Comparison

Feature	TaskSphere	Trello	Jira	G-Classroom	MS Planner	Asana
University Role Hierarchy	✓	✗	Partial	✗	✗	✗
Enforced Status Workflow	✓	✗	✓	✗	✗	Paid
Mandatory Evidence Upload	✓	✗	✗	✗	✗	✗
Auto Deadline Email Reminder	✓	✗	✗	✗	✗	Paid
Activity Audit Log	✓	✗	Paid	✗	✗	Paid
Session-Based Auth (no cloud)	✓	✗	✗	✗	✗	✗
BCrypt Password Security	✓	N/A	N/A	N/A	N/A	N/A
First-Login Pwd Enforcement	✓	✗	✗	✗	✗	✗
Team Leader Designation	✓	Partial	Partial	✗	✗	Partial
Free & Self-Hosted	✓	Partial	✗	✗	Partial	✗
No Third-Party Cloud Dep.	✓	✗	✗	✗	✗	✗

Table 2.1: Feature Comparison – TaskSphere vs Existing Systems

2.3 Limitations of Existing Solutions

- All reviewed tools are horizontal general tools that do not contain any pre-installed model for the (“three-tier”) university hierarchy with administrators, teachers, students.
- No tools impose a set of task status transitions for mandatory forward movement. On all of them, any member can push any task up/down any time.
- None of the reviewed tools would mandate computer-generated evidence of a completed task to mark a task as completed.

- The deadline reminder emails are automated and customized to academic teams in the free tier of all the tools reviewed.
- No tools offer a forced password change that works for onboarding at the University's first login.
- Thus, all reviewed tools rely on third party cloud providers and are thus unsuitable for use inside the university cloud infrastructure.
- There are also a lack of integrated review approval and re-submission workflows that are tailored for academic project monitoring and evidence validation for most of the current systems.

2.4 Research Gap

Based on the preceding review, none of the currently available general-purpose or education specific tools offers all of the above in a single self-contained, open source application. TaskSphere aims to entirely fill this void without need for cloud accounts, paid subscription or complicated setup.

In addition to this, TaskSphere is brought to you with new teacher controlled approval and reupload workflows, notifications in a central place and activity accountability options specifically for university project supervision environments.

CHAPTER 3: SYSTEM REQUIREMENTS

3.1 Hardware Requirements

Component	Minimum Specification
Processor	Intel Core i3 / AMD Ryzen 3 (or equivalent), 1.8 GHz or above
RAM	4 GB minimum; 8 GB recommended for smooth development and testing
Storage	20 GB available disk space for application JAR, MySQL data, and uploaded screenshot files
Network	Active internet connection required for SMTP email dispatch via Gmail
Display	1280 × 720 resolution minimum for dashboard usability
Server (Deployment)	Any machine capable of running JDK 21 and MySQL 8 simultaneously

Table 3.1: Hardware Requirements

TaskSphere's hardware needs are minimal, meaning that the system can be installed within the infrastructure of an existing university department laboratory or faculty workstation without the need for enterprise class hardware.

3.2 Software Requirements

Software / Tool	Purpose and Version
Java Development Kit (JDK 21)	Application runtime; Java 21 LTS required by Spring Boot 3.3.6
Spring Boot 3.3.6	Core MVC framework; embedded Apache Tomcat server; auto-configuration
Spring Boot Starter JDBC	Provides JdbcTemplate for all SQL operations (no ORM used)
Spring Security Crypto	BCryptPasswordEncoder for one-way password hashing only
Thymeleaf 3.x	Server-side HTML template engine for all view rendering
MySQL 8.x	Relational database; schema initialised via schema.sql on startup
Spring Boot Starter Mail	JavaMailSender for SMTP email dispatch (reminders, password reset)
Spring Boot Starter Validation	Bean Validation annotations (@NotBlank, @NotNull) on form classes

Software / Tool	Purpose and Version
Apache Maven 3.x	Build tool; dependency management via pom.xml
IntelliJ IDEA 2024	Development IDE
Google Chrome / Firefox	Browser for manual testing and UI validation
MySQL Workbench / CLI	Database inspection and query validation during testing
Git 2.x	Version control

Table 3.2: Software Requirements

3.3 Functional Requirements

FR ID	Requirement	Role	Description
FR01	Secure Login	All Roles	Authenticate via role, identifier (email or enrollment number), and BCrypt-verified password. Store UserSession in HTTP session.
FR02	First-Login Password Change	Teacher, Student	Redirect user to password change form if first_login_required = true.
FR03	Password Reset via Email	All Roles	Generate UUID token, store with 15-min expiry, email link, validate and consume on reset.
FR04	Create Teacher Account	Super Admin	Create teacher with name, email, contact, temporary password (BCrypt-hashed).
FR05	Create Student Account	Super Admin	Create student with name, email, enrollment number, program, contact. Initial password = BCrypt(enrollment_number).
FR06	Edit / Delete Users	Super Admin	Update name/email/contact of teachers; delete teacher or student records.
FR07	Admin Dashboard Summary	Super Admin	Display total counts of super admins, teachers, students, and teams by status (global).
FR08	Create Project Team	Teacher	Create team with unique name (auto-generated as TEAM-XXXX), task name, deadline, and initial members via dynamic form.
FR09	Manage Team Members	Teacher	Add, remove, or update members; enforce single-team rule via UNIQUE constraint on student_id in team_members.
FR10	Designate Team Leader	Teacher	Set is_leader = true for one member; clear previous leader via clearLeader() before setting new one.
FR11	Update Task / Deadline	Teacher	Edit task name or deadline (must be future date) for an existing team.
FR12	Delete Team	Teacher	Delete team; cascade deletion removes all team_members and activity_logs records.
FR13	Configure Reminder Days	Teacher	Update reminder_days in reminder_settings table via ReminderRepository.
FR14	View Team and Members	Student	View assigned teams, task name, deadline, teacher name, status, and all members.

FR ID	Requirement	Role	Description
FR15	Update Task Status	Team Leader	Advance status PENDING → IN_PROGRESS → COMPLETED; enforced by TeamStatus.canTransitionTo().
FR16	Upload Completion Screenshot	Team Leader	Upload PNG/JPG/JPEG/GIF/WEBP file (max 10 MB) when marking task as COMPLETED; stored with UUID filename.
FR17	Automated Email Reminder	System (Scheduler)	Daily at 18:00 via @Scheduled cron: send reminder emails to all team members when deadline – reminder_days ≤ today, if reminder_sent = false.
FR18	Activity Logging	System	Record actor_role, actor_display_name, action_type, description, created_at for all significant operations.
FR19	Team Assignment Email	System	Send email notification to all team members on team creation or update.
FR20	Teacher Review Workflow	Teacher	Approve completed submissions or request re-upload with feedback.
FR21	Centralized Notifications	System	Display review updates, task alerts, and deadline reminders on dashboards.
FR22	AJAX Duplicate Validation	Teacher	Validate student email and enrollment number in real time during team creation.
FR23	Re-upload Workflow	Student	Allow team leader to upload revised completion proof after teacher rejection.

Table 3.3: Functional Requirements

3.4 Non-Functional Requirements

- **Security:** All passwords are saved as BCrypt (bcryptpasswordencoder from spring-security-crypto). Server.servlet.session.cookie.http-only=true is set in the server HTTP session cookie. server.servlet.session.timeout=30m (the session timeout is set to 30 minutes.). Use role guard check for each controller method (using BaseController).hasRole().
- **Data Integrity:** The referential integrity constraints created in schema.sql allow for the following: teams-foreign-key-teachers, team_members-foreign-key-teams, team_members-foreign-key-students, activity_logs-foreign-key-teams. Note that when a team is deleted, there will be no orphan records because cascade delete is applied to team_members and activity_logs.
- **Reliability:** @Transactional annotation on TeamService.createTeam() and TeamService.updateTeamFull() ensures that all database writes within a team creation or update operation are committed atomically or rolled back entirely on failure.

- Scalability: The stateless Spring MVC controller layer is horizontally scalable behind a load balancer, as long as the persistence of the session is done in an external way, e.g. by the JdbcTemplate which is not responsible for the state of the entity.
- Maintainability: The package separation (controller -> service -> repository -> model -> form -> config) is well defined, and there are no ORM annotations, which make the application easy to read and edit.
- Portability: You can host this application on any server with JDK 21 and MySQL 8. There won't be any outside cloud services, message brokers, or proprietary API needed.
- File Upload Constraints: Multipart file size limited to 10 MB (spring.servlet.multipart.max-file-size=10MB). Accepted image formats validated by extension: .png, .jpg, .jpeg, .gif, .webp, .pdf, .docx.
- Server Port: Application is running on the port 9090 (server.port=9090) and the server time zone is Asia/Kolkata.
- Usability: Students and teachers will find the user interface more user-friendly due to the functionalities created by Bootstrap 5 including responsive column layouts, fixed dashboard panel area, and centralized notification areas.
- Maintainability of UI: Thymeleaf templates are modular and reusable, enabling fast UI enhancements as well as future ease of maintenance.
- Availability: The application is deployed and started in a simple and reliable manner for institutional usage as it is based on an embedded Tomcat server, where operator are grouped via Spring Boot.

CHAPTER 4: SYSTEM DESIGN

4.1 System Architecture

TaskSphere is a three-tier web application following monolithic architecture for web applications:

4.1.1 Presentation Tier

The presentation layer is made of completely HTML-based Thymeleaf templates, delivered on the fly using the embedded Apache Tomcat server of Spring Boot. Pages are delivered from the server using thymeleaf: The Spring controller loads data into the Model object that comes from the service layer and the thymeleaf engine constructs the complete page with the URL template and fills the model object with the data. No client-side rendering framework, only HTML is sent to the browser. Only two exceptions are made when using vanilla JavaScript: the AJAX check-email and check-enrollment of student identifiers as they are typed as the user types them into the team creation form; and basic client-side logic for checking and verifying form data. The `src/main/resources/static` directory is used for static resources served (CSS, JavaScript, Files). Server side uploads directory mapping/upload of uploaded files are served from `/uploads/**` via `WebConfig.addResourceHandlers()`.

4.1.2 Application Tier

The application tier is a Spring Boot 3.3.6 application that is broken into four layers:

- **Controllers:** All incoming HTTP GET and POST requests are handled by these 6 classes: `HomeController`, `AuthController`, `AdminController`, `TeacherController`, `StudentController`, and the abstract `BaseController` (which checks roles). Use `hasRole()` to assess if a user has a certain role and leave all business logic to service classes. No one other than controllers will touch repositories directly.
- **Services:** All business rules are defined in the services such as `AuthenticationService`, `PortalService`, `TeamService`, `EmailService`, `FileStorageService`, `PasswordService`, `PasswordResetService`, `DeadlineReminderScheduler`, `BootstrapService`. The `TeamService` methods that involve multiple repository operations are marked with `@Transactional`.

- **Repositories:** All SQL in the system is encapsulated in 8 of those classes (SuperAdminRepository, TeacherRepository, StudentRepository, TeamRepository, TeamMemberRepository, ActivityLogRepository, PasswordResetRepository, ReminderRepository) definitions of repository. There is one JdbcTemplate injected per repository. All database operation calls are implemented using JdbcTemplate.query(), JdbcTemplate.queryForObject(), and parameterised PreparedStatements in JdbcTemplate.update(). The project contains no ORM, no Hibernate and no JPA entity class.
- **Models / Forms:** Data is carried between the different layers using the Models / Forms classes which are java record classes (Team, Teacher, Student, SuperAdmin, TeamMember, TeamMemberView, TeamView, ActivityLog, UserSession, UiNotification, DashboardSummary, PasswordResetToken). Controller methods include @ModelAttribute objects such as Form classes: LoginForm, CreateTeacherForm, CreateStudentForm, CreateTeamForm, UpdateTeamForm, UpdateTeamStatusForm, ChangePasswordForm, EditTeacherForm, MemberDto that support Bean Validation.

4.1.3 Data Tier

The data tier is a MySQL 8 relational database, called “tasksphere_db”, automatically created on first start up, if the database does not exist (createDatabaseIfNotExist=true in the JDBC URL). The schema in schema.sql is defined with Spring's SQL initialisation mechanism (spring.sql.init.mode=always). The database includes eight tables having the relevant primary key, foreign key constraint, unique constraint and cascade delete attributes from schema.sql. No ORM mapping, no hibernate DDL, the schema is completely defined in the schema.sql.

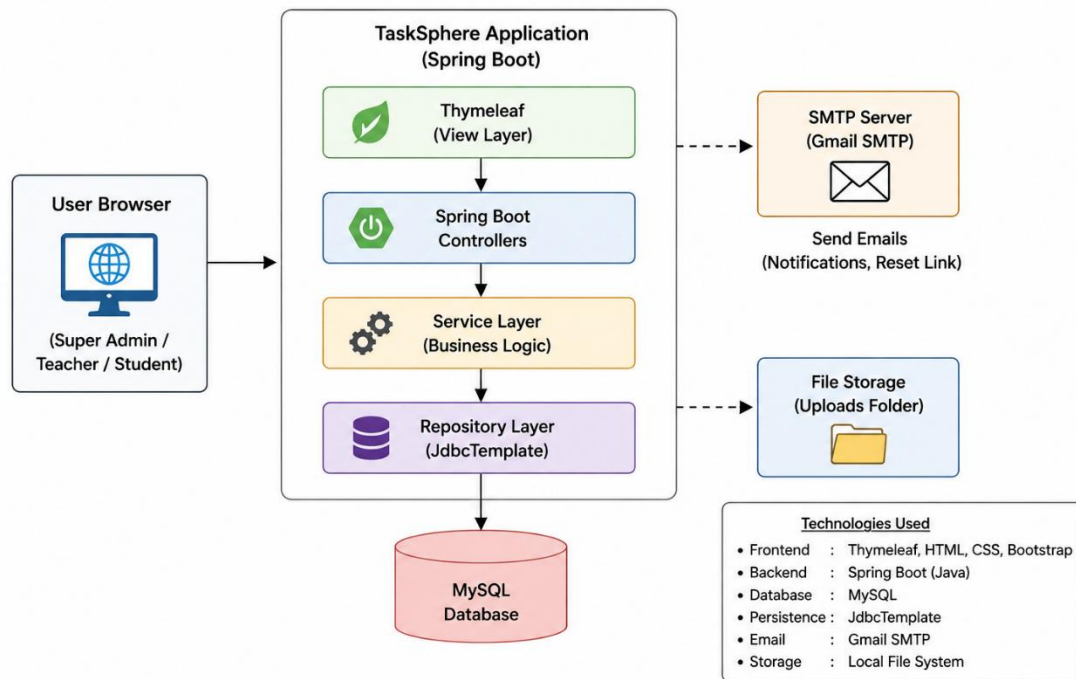


Figure 4.1: System Architecture Diagram

Figure 4.1: Three-Tier System Architecture of TaskSphere

Three levels architecture used in the TaskSphere is suitable and effective separation of concerns in the presentation layer, business logic layer, and data access layer. By this architectural track, future feature enhancements and deployments are simplified, along with maintainability, scalability and modularity.

4.2 Data Flow Diagrams

4.2.1 Context-Level DFD (Level-0)

The Level-0 DFD is the overall DFD for the TaskSphere system, and shows three external entities and one external service interacting with the system:

- Super Admin → System: Request for adding/updating of credentials, account deletion.
- Teacher → System: create new team, assign tasks, manage team members, designate members as leaders, set reminders, approve / reject task, change passwords.
- Student (Team Leader) → System: Task status change, upload file of team project report, change password
- System → Emails (Gmail SMTP) will automatically notify team members of deadlines, password reset, and team assignment.

- System → All Roles: presented dashboards, logs of activity, notification alerts, success/error messages.

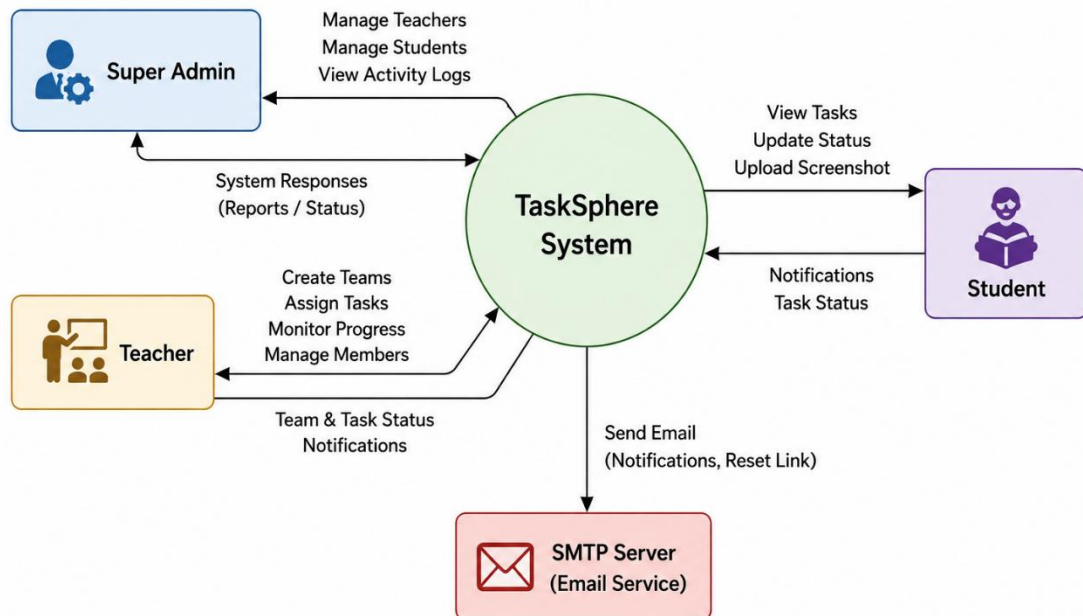


Figure 4.2: DFD Level 0 (Context Diagram)

Figure 4.2: Context-Level Data Flow Diagram (DFD Level-0)

The general context of how the user interacts with the TaskSphere system is described in the Level-0 DFD. It shows how voice mail information is exchanged between the centralizations ITM and students and administrators, and email is conveyed through the SMTP service.

4.2.2 Level-1 DFD

The Level-1 DFD breaks the TaskSphere system down into its six major functional processes (Data Stores):

- Process 1 – Authentication: Looks at the facts of the form submitted when a user logs in; depending on the logon role selected, queries the appropriate data store (Student, Teacher or super_admins); checks if the logon facts match a specific UserSession BCrypt hash; creates and stores a UserSession in the HTTP Session and routes to the appropriate logon role Dashboard.
- Process 2 – User Management: This process is run by the Super Admin. Reads and writes to the super_admins, teachers, and students data stores. Manages

creating modifications and deleting accounts (accounts are hashed with BCrypt password hashing).

- Process 3 – Team Management: Informed by the Teacher. Presents data to the teams data store and team_members data store. Accesses student data store to verify and solve member identifiers. Writes to activity_logs data store after all important activities.
- Process 4 – Task Status Management: Team Leader (Student) Reads information about teams and membership, validates Transition to or from Team, updates information to the Teams data store, writes new information to activity_logs.
- Process 5 – Email Notification occurs when it is activated by cron (synched by the DeadlineReminderScheduler), when a team is created or updated, and when the password reset flow is executed. Records data from team, team_members, students, password_reset_token data stores. Sends e-mail using external mail service (SMTP).
- Process 6 – Activity Logging is called by Processes 3 & 4. Composes to the activity_logs data store. Refers to the reading when creating the Teacher and Student dashboard.

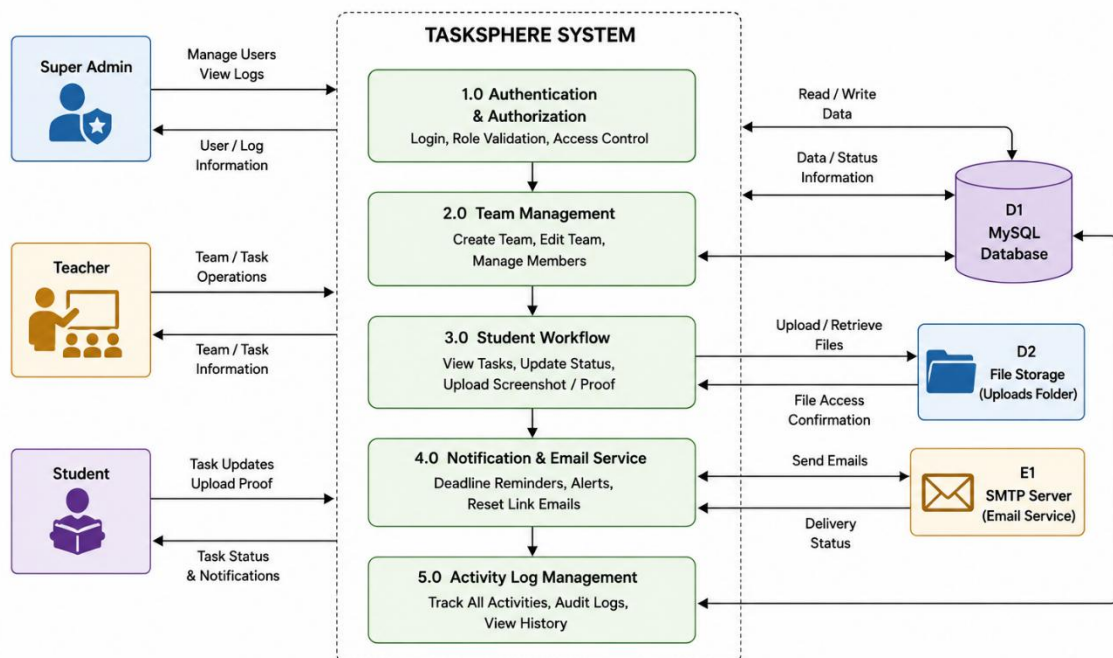


Figure 4.3: DFD Level 1

Figure 4.3: Level-1 Data Flow Diagram

The internal operations of TaskSphere are broken down into a number of functional modules in the Level-1 DFD: authentication, team management, team workflow processing, handling notifications, and logging activities.

4.3 Use Case Diagram

There are four actors, with their corresponding use cases, identified in the Use Case Diagram:

- Super Admin: Login, Logout, Reset Password, Create Teacher Account, Edit Teacher Account, View Student Status.
- Teacher: Login, Logout, Reset Password, Manage password, Create Team, View Team Detail, Edit Team (task/deadline), Add Member, Remove Member, Designate Team Leader, Delete Team, Configure Reminder Days, View Teacher Dashboard, View activity Log.
- Student (All): Log-in, Log-out, reset password, change password, view student dashboard, and view activity log, view status and upload files.
- Student (Team Leader only): Update Task Status (PENDING → IN_PROGRESS → COMPLETED), Upload Completion File.
- System: Send Deadline Reminder Email (scheduled), Send Team Assignment Email (triggered by create/update), Purge Expired Password Reset Tokens (scheduled every 10 minutes), approve / reject task file.

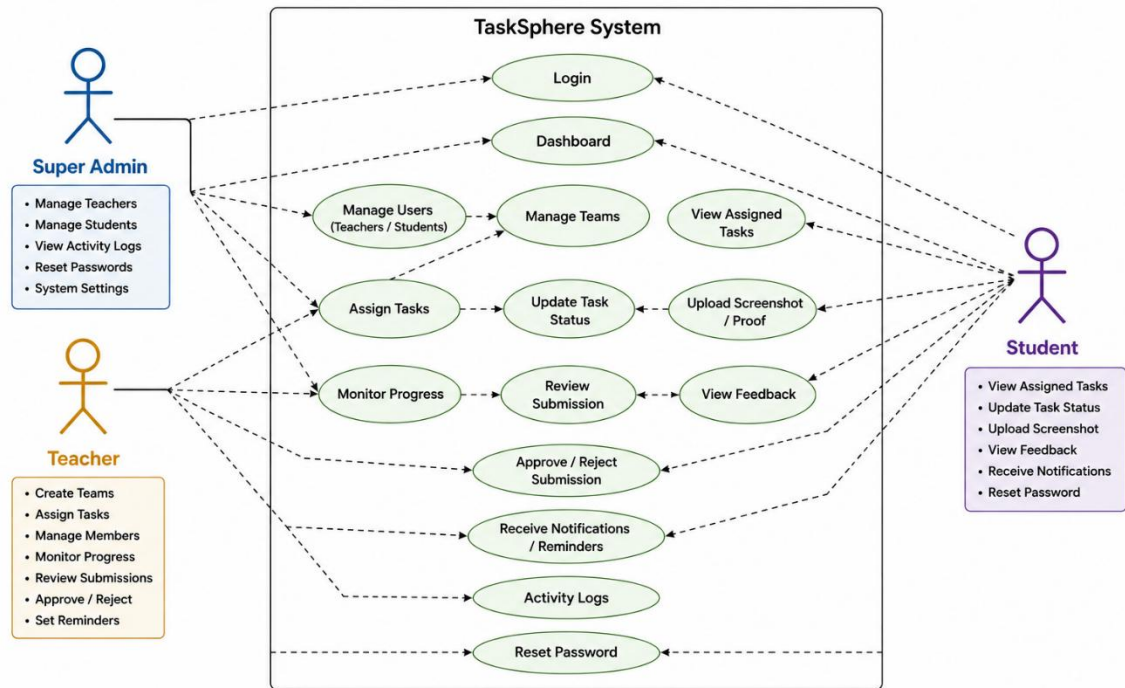


Figure 4.4: Use Case Diagram

Figure 4.4: Use Case Diagram – All Roles

The Use Case Diagram shows how various User's of the system participate in relationships with each other and TaskSphere provides functionality. There are role-based permissions which are enforced by the role-based access control mechanism, each role functioning within controlled permissions.

4.4 ER Diagram and Database Schema

The TaskSphere entity-relationship model is structured around the central teams entity, which acts as the linking entity between the teachers and students domains:

- super_admins is a standalone entity with no foreign keys. It stores credentials for the system administrator role.
- teachers has a one-to-many relationship with teams. One teacher can own multiple teams; each team belongs to exactly one teacher (FK: teacher_id → teachers.id).
- students has a many-to-many relationship with teams, mediated by the team_members junction table. The constraint uq_student (UNIQUE on student_id in team_members) enforces that a student can belong to at most one team at any time.
- teams is the central entity. It stores the task name, deadline, status, screenshot_path, and reminder_sent flag, in addition to the teacher_id foreign key.

- `team_members` stores the `team_id` and `student_id` pair (both foreign keys), with a composite unique constraint `uq_team_member` (`team_id`, `student_id`) preventing duplicate membership. The `is_leader` boolean identifies the designated team leader.
- `activity_logs` stores an audit record for every significant action. It references teams via a foreign key with ON DELETE CASCADE, so all logs for a team are automatically removed when the team is deleted.
- `password_reset_token` stores email, token, and expiry_time. It has no foreign key to other tables. A MySQL EVENT named `delete_expired_tokens` runs every 10 minutes to purge expired rows.
- `reminder_settings` is a single-row configuration table holding the global `reminder_days` integer. Row 1 is inserted by `data.sql` at startup and updated by `ReminderRepository.updateReminderDays()`.

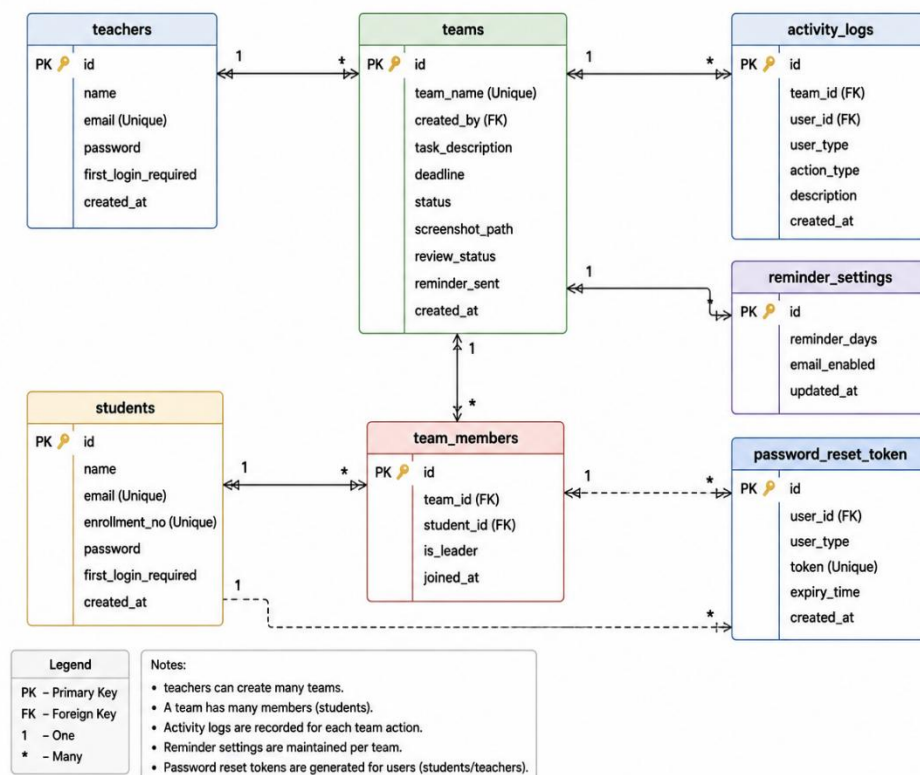


Figure 4.5: ER Diagram

Figure 4.5: Entity Relationship (ER) Diagram

4.5 Database Tables

Schema.sql contains all the tables definitions, and will create all the needed tables with IF NOT EXISTS guards, thus the schema is idempotent against restart. Below are the full TaskSphere DBs:

Table 4.1: super_admins

Column Name	Data Type	Constraint	Description
id	BIGINT	PRIMARY KEY, AUTO_INCREMENT	Unique surrogate key for the super admin record.
name	VARCHAR(120)	NOT NULL	Full name of the super administrator.
email	VARCHAR(180)	NOT NULL, UNIQUE	Login identifier and communication address.
password_hash	VARCHAR(255)	NOT NULL	BCrypt-hashed password (60-character output).
contact_number	VARCHAR(10)	NOT NULL	10-digit mobile contact number.

Table 4.1: super_admins – System Administrator Credentials

Table 4.2: teachers

Column Name	Data Type	Constraint	Description
id	BIGINT	PRIMARY KEY, AUTO_INCREMENT	Unique surrogate key for the teacher record.
name	VARCHAR(120)	NOT NULL	Full name of the teacher.
email	VARCHAR(180)	NOT NULL, UNIQUE	Login identifier; must be globally unique.
password_hash	VARCHAR(255)	NOT NULL	BCrypt-hashed password.
contact_number	VARCHAR(10)	NOT NULL	10-digit mobile contact number.
first_login_required	BOOLEAN	NOT NULL, DEFAULT true	Set true on creation; cleared after first password change.

Table 4.2: teachers – Faculty Account Records

Table 4.3: students

Column Name	Data Type	Constraint	Description
id	BIGINT	PRIMARY KEY, AUTO_INCREMENT	Unique surrogate key for the student record.
name	VARCHAR(120)	NOT NULL	Full name of the student.

Column Name	Data Type	Constraint	Description
enrollment_number	VARCHAR (40)	NOT NULL, UNIQUE	University enrollment number; used for login and as initial password seed.
password_hash	VARCHAR (255)	NOT NULL	BCrypt hash of the initial password (enrollment number) or user-chosen password.
program	VARCHAR (160)	NOT NULL	Academic programme name (e.g., B.Tech CSE).
email	VARCHAR (180)	NOT NULL, UNIQUE	Email address for notifications and alternate login.
contact	VARCHAR (10)	NOT NULL	10-digit mobile contact number.
first_login_required	BOOLEAN	NOT NULL, DEFAULT true	Cleared after the student changes their initial password.

Table 4.3: students – Student Account Records

Table 4.4: teams

Column Name	Data Type	Constraint	Description
team_id	BIGINT	PRIMARY KEY, AUTO_INCREMENT	Unique surrogate key; used to generate auto-name TEAM-XXXX.
team_name	VARCHAR (40)	NOT NULL, UNIQUE	Auto-generated as TEAM-0001, TEAM-0002, etc. via formatted(teamId).
task_name	VARCHAR (180)	NOT NULL	Description of the project task assigned to the team.
teacher_id	BIGINT	NOT NULL, FK → teachers(id)	References the creating teacher. Not cascade-deleted.
deadline	DATE	NOT NULL	Task submission deadline; must be a future date on creation.
status	VARCHAR (30)	NOT NULL	TeamStatus enum value: PENDING, IN_PROGRESS, or COMPLETED.
screenshot_path	VARCHAR (255)	NULLABLE	Relative filename of the uploaded completion screenshot (set when COMPLETED).
reminder_sent	BOOLEAN	DEFAULT false	Set true after reminder email dispatched; prevents duplicate sending.

Table 4.4: teams – Project Team Records

Table 4.5: team_members

Column Name	Data Type	Constraint	Description
id	BIGINT	PRIMARY KEY, AUTO_INCREMENT	Unique surrogate key for the membership record.
team_id	BIGINT	NOT NULL, FK → teams(team_id) ON DELETE CASCADE	References the associated team. Cascade delete removes members when team is deleted.
student_id	BIGINT	NOT NULL, FK → students(id), UNIQUE (uq_student)	References the student. UNIQUE constraint enforces one-team-per-student rule.
is_leader	BOOLEAN	NOT NULL, DEFAULT false	True for exactly one member — the designated team leader.

Table 4.5: team_members – Team Membership Junction Table

Table 4.6: activity_logs

Column Name	Data Type	Constraint	Description
id	BIGINT	PRIMARY KEY, AUTO_INCREMENT	Unique surrogate key for the log entry.
team_id	BIGINT	NOT NULL, FK → teams(team_id) ON DELETE CASCADE	References the team the action relates to. Cascade delete removes all logs when team is deleted.
actor_role	VARCHAR(30)	NOT NULL	Role of the actor: TEACHER or STUDENT.
actor_display_name	VARCHAR(180)	NOT NULL	Display name of the user who performed the action.
action_type	VARCHAR(60)	NOT NULL	Category of action: TEAM_CREATED, MEMBER_REMOVED, LEADER_CHANGED, STATUS_UPDATED, etc.
description	VARCHAR(255)	NOT NULL	Human-readable sentence describing the specific action performed.
created_at	TIMESTAMP	NOT NULL, DEFAULT CURRENT_TIMESTAMP	Server timestamp at the moment the action was logged.

Table 4.6: activity_logs – Audit Trail

Table 4.7: password_reset_token

Column Name	Data Type	Constraint	Description
id	BIGINT	PRIMARY KEY, AUTO_INCREMENT	Unique surrogate key.
email	VARCHAR(255)	NOT NULL	Email address of the user requesting the reset.
token	VARCHAR(255)	NOT NULL, UNIQUE	UUID-based random token included in the reset link.
expiry_time	DATETIME	NOT NULL	Expiry set to LocalDateTime.now().plusMinutes(15) at creation.

Table 4.7: password_reset_token – Password Reset Tokens

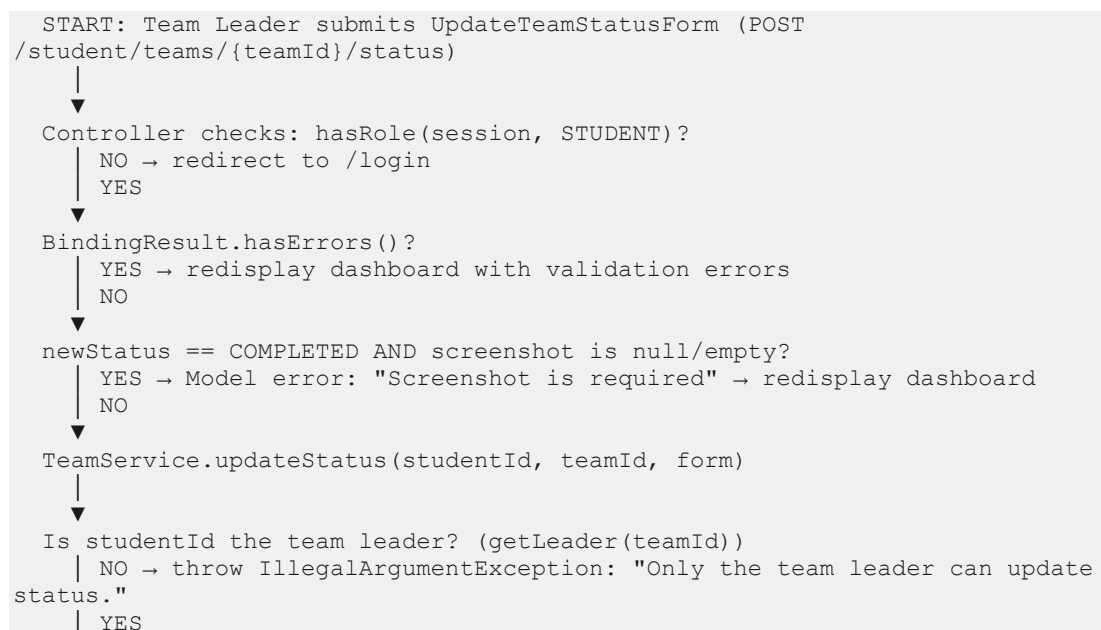
Table 4.8: reminder_settings

Column Name	Data Type	Constraint	Description
id	BIGINT	PRIMARY KEY, AUTO_INCREMENT	Surrogate key; row 1 is seeded by data.sql.
reminder_days	INT	NOT NULL	Number of days before deadline at which reminder emails are dispatched.

Table 4.8: reminder_settings – Global Scheduler Configuration

4.6 Task Workflow Flowchart

The full logic process that is executed when the team leader updates the task status using Student: StudentController is outlined below in a flowchart:



```

TeamStatus.canTransitionTo(nextStatus)?
| NO → throw IllegalArgumentException: "Status can only move forward one
step."
| YES
    ↓
    nextStatus == COMPLETED?
    | YES → FileStorageService.storeCompletionScreenshot(file, teamId,
studentId)
    | NO → screenshotPath = existing value (null or previous)
        • Validate extension ∈ { .png, .jpg, .jpeg, .gif, .webp }
        • Generate filename: team-{id}-student-{id}-{UUID}.ext
        • Files.copy() to uploadRoot (server-side directory)
        • Return stored filename → screenshotPath
    ↓
    TeamRepository.updateStatus(teamId, nextStatus, screenshotPath)
    ↓
    → UPDATE teams SET status = ?, screenshot_path = ? WHERE team_id = ?
    ↓
    ActivityLogRepository.create(teamId, "STUDENT", studentName,
"STATUS_UPDATED", description)
    ↓
    → INSERT into activity_logs ...
    ↓
    Controller: redirectWithSuccess → "redirect:/student/dashboard"
    ↓
    END

```

TaskSphere - Task Lifecycle Workflow

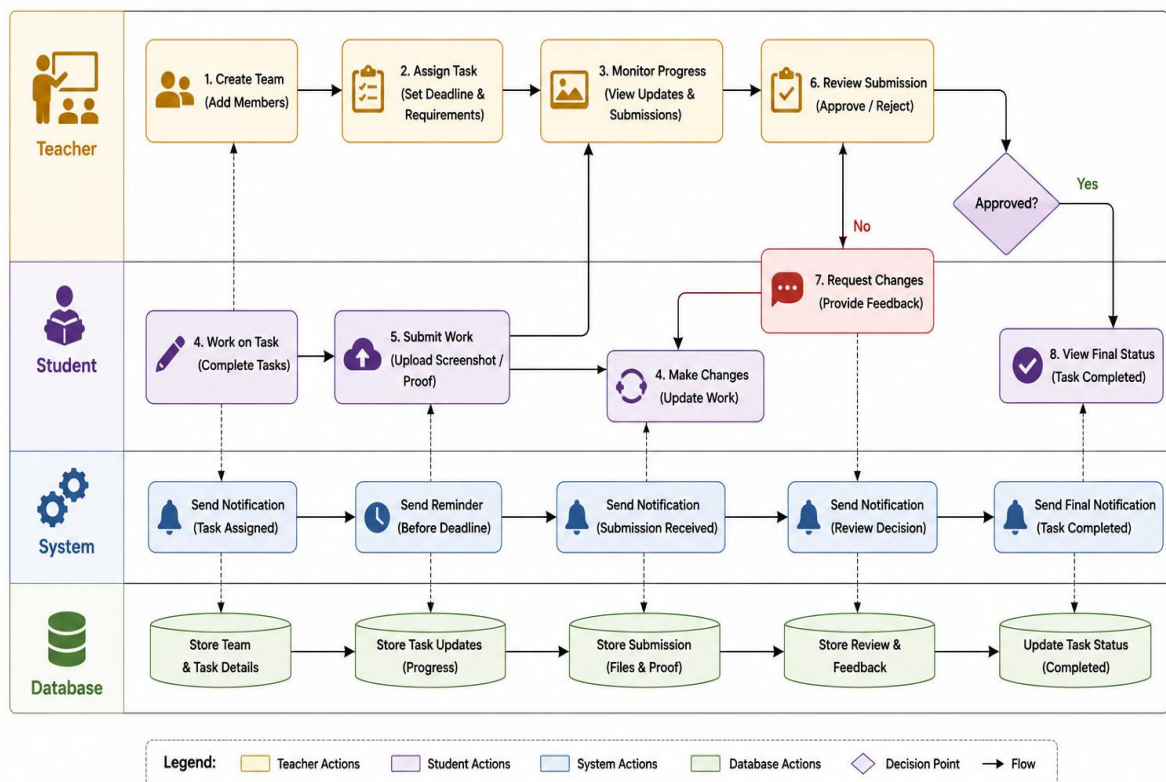


Figure 4.6: Task Lifecycle Workflow Diagram

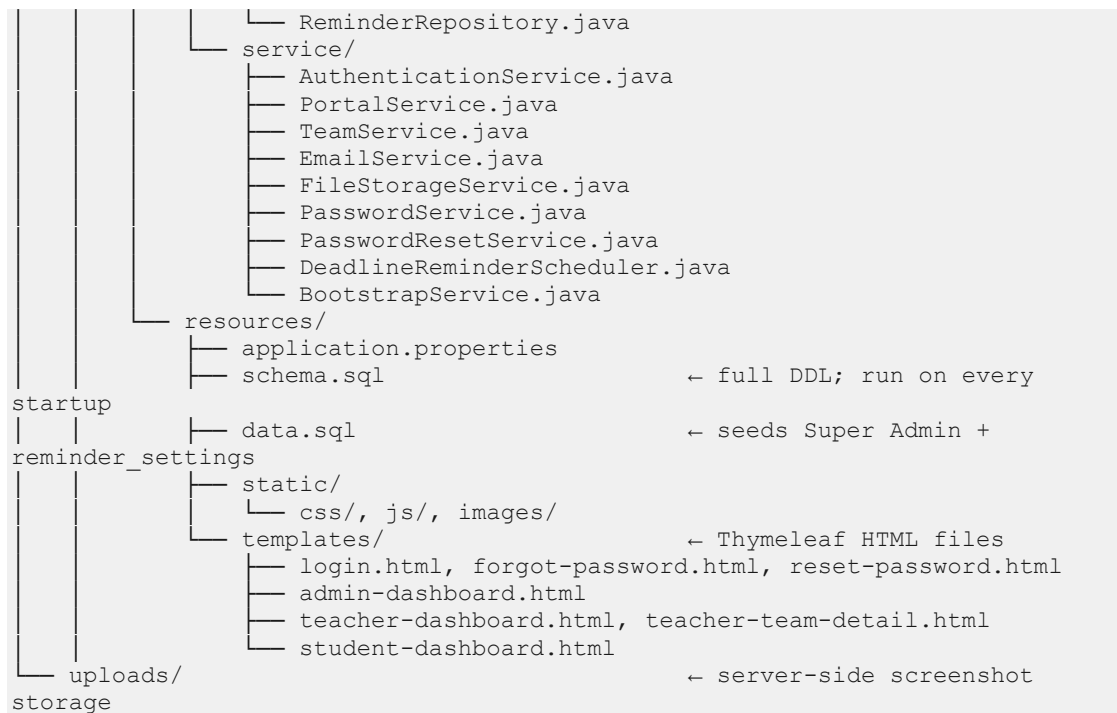
Figure 4.6: Task Status Update Flowchart

CHAPTER 5: IMPLEMENTATION

5.1 Project Structure

The project uses the "Maven" standard directory structure for development projects. All the application source code is housed in `src/main/java/com/tasksphere/` and is structured in these packages:

TaskSphere/	
pom.xml	
src/	
main/	
java/com/tasksphere/	
TaskSphereApplication.java	← entry point
(@SpringBootApplication)	
config/	
WebConfig.java	← resource handler for
/uploads/**	
controller/	
BaseController.java	← session helpers
HomeController.java	← "/" → role-based redirect
AuthController.java	← login, logout,
forgot/reset password	
AdminController.java	← super admin dashboard and
CRUD	
TeacherController.java	← team management, AJAX
checks	
StudentController.java	← dashboard, status update
form/	
LoginForm.java	
CreateTeacherForm.java	
CreateStudentForm.java	
CreateTeamForm.java	← List<MemberDto> +
leaderIndex	
UpdateTeamForm.java	
UpdateTeamStatusForm.java	← status + MultipartFile
screenshot	
ChangePasswordForm.java	
EditTeacherForm.java	
MemberDto.java	
model/	
Role.java	← enum: SUPER_ADMIN, TEACHER,
STUDENT	
TeamStatus.java	← enum with
canTransitionTo() logic	
UserSession.java	← record stored in HTTP
session	
Team.java, Teacher.java, Student.java, SuperAdmin.java	
TeamMember.java, TeamMemberView.java, TeamView.java	
ActivityLog.java, DashboardSummary.java	
UiNotification.java, PasswordResetToken.java	
repository/	
SuperAdminRepository.java	← JdbcTemplate-based SQL
TeacherRepository.java	
StudentRepository.java	
TeamRepository.java	
TeamMemberRepository.java	
ActivityLogRepository.java	
PasswordResetRepository.java	



5.2 Technology Stack

Technology / Library	Role in TaskSphere
Java 21 (LTS)	Application language. Records, switch expressions, and text blocks used throughout.
Spring Boot 3.3.6	MVC framework; embedded Tomcat; auto-configuration; <code>@Scheduled</code> support.
spring-boot-starter-jdbc	Provides <code>JdbcTemplate</code> . All database access uses parameterised SQL via <code>JdbcTemplate</code> . No ORM or JPA entities are used.
spring-security-crypto	Provides <code>BCryptPasswordEncoder</code> used exclusively in <code>PasswordService</code> for hashing and verification.
Thymeleaf 3.x	Server-side HTML template engine. All pages rendered by the Spring controller via the Thymeleaf template resolver.
spring-boot-starter-mail	Provides <code>JavaMailSender</code> used by <code>EmailService</code> to send plain-text emails via Gmail SMTP (port 587, STARTTLS).
spring-boot-starter-validation	Bean Validation (<code>@NotBlank</code> , <code>@NotNull</code> , <code>@Size</code>) on form classes; <code>BindingResult</code> used in controllers.
MySQL 8.x (mysql-connector-j)	Production relational database. Schema managed by <code>schema.sql</code> ; data seeded by <code>data.sql</code> .
HTML5 / CSS3 / Vanilla JS	Frontend: Thymeleaf renders complete HTML; JavaScript handles AJAX identifier checks and UI interactions.
Apache Maven 3.x	Dependency management and build lifecycle (<code>mvn clean package</code>).

Technology / Library	Role in TaskSphere
spring-boot-starter-data-jpa	Listed in pom.xml but no JPA entities or repositories are used. JdbcTemplate is used for all data access.

Table 5.1: Technology Stack

5.3 Module-wise Implementation

5.3.1 Session-Based Authentication

When it comes to authentication in TaskSphere, Spring Security's filter chain isn't used. Instead, it is manually done by using AuthController, AuthService and abstract BaseController class.

On the user submitting the login form (POST /login) AuthController presents the user with a LoginForm (and validates it by Bean Validation), it resolves the role from the populist string literal and calls AuthenticationService.authenticate(role, identifier, password). From a java switch expression, the request is directed to the right repository in the case of inside authenticate. The identifier should be the registered email address for SUPER_ADMIN and TEACHER. For STUDENT, it determines if the identifier contains the "@": if it does, it will query to StudentRepository.If yes it will findByEmail(); else it will query StudentRepository.findByEnrollmentNumber() (converting the input to uppercase). The found user object is passed through PasswordService.matches(rawPassword, storedHash) which is the call to BCryptPasswordEncoder.matches. A UserSession Java record, containing the UserId, the User's Principal, the User's displayName, the User's Role, and the firstLoginRequired flag is stored in the HTTP session under the key TASKSPHERE_USER.

The result of execution of any of the controller methods marked protected starts by invoking BaseController.hasRole(session, expectedRole). This approach will get the UserSession from the session, and verify its role. RedirectWithError() redirects to /login if the session doesn't exist or if there is a mismatch between the specified role and the actual one, including a message. This mechanism offers access control based on roles when there are no URL-based interceptors provided by Spring Security.

Session configuration is applied in application.properties: timeout is 30 minutes (server.servlet.session.timeout=30m) and cookies are marked HTTP-only

(`server.servlet.session.cookie.http-only=true`), preventing JavaScript from reading the session cookie.

5.3.2 JdbcTemplate Repositories

JdbcTemplate is used by all the 8 repository classes in TaskSphere to handle queries to the database. This is a conscious design decision – you get full control over the SQL that is sent to the database, but not the abstraction magic or configuration complexity of an ORM like Hibernate. All repositories are Spring-beans annotated with the `@Repository` annotation and contain only one `@Autowired JdbcTemplate`.

Every repository has its own private static final `RowMapper<T>` which it's responsible for converting a JDBC `ResultSet` row to the applicable Java record. For example, `TeamRepository`. A team represents the mapping for the `team_id`, `team_name`, `task_name`, `teacher_id`, `deadline`, `status`, `screenshot_path` and `reminder_sent` fields for the team. For operations where the auto-generated primary key is required (like `TeamRepository.create()`), pass a `PreparedStatementCreator` into the `Statement`. Since there are no keys to retrieve, you need to use `RETURN_GENERATED_KEYS` to get them and retrieve a `GeneratedKeyHolder` so you can retrieve the generated `team_id`. All other writes take a more straightforward `JdbcTemplate.update(sql, args...)` form. All reads return a `List` by using `JdbcTemplate.query(sql, ROW_MAPPER, args)` stream. The `convertToOptional<T>` method used to be called `findFirst()`, but has changed to `Optional<T>` for a single result.

5.3.3 Super Admin Module (AdminController + PortalService)

This module is added by Super Admin. It is a Super Admin Module (`AdminController + PortalService`) 5.3.3 The `AdminController.dashboard()` method (`GET /admin/dashboard`) generates the Super Admin dashboard. The controller calls to `PortalService` to set four count statistics: `totalSuperAdmins`, `totalTeachers`, `totalStudents` and `TeamService`. Return a summary of the totals and counts of teams grouped by status for a `DashboardSummary`. Return a total teams and counts by status for a `DashboardSummary`. The complete list of teachers and students (with team status) is also sent to the model. These are displayed by the Thymeleaf template as summary cards and tabbed tables. These are shown through the Thymeleaf template in summary card or tabbed tables..

Teacher creation (POST /admin/teachers): Reads a CreateTeacherForm, validates it on-the-fly using the Bean Validation, and uses the PortalService. It hashes the transient password using BCrypt prior to a call to TeacherRepository.create(), with this call being done through createTeacher(). When using the JDBC driver, the driver will throw a DuplicateKeyException; PortalService will catch this exception and pass it back to the client as an IllegalArgumentException message that is more user friendly. The same applies to password set for a student's creation, except that the initial password is automatically the hash of the enrollment number using BCrypt.

Another way for the Admin to update his/her profile is to use POST /admin/update-profile which is accessed by PortalService, and provides updating the profile's email (and optionally the new password). It will first check against the existing password before making any changes, as in the updateAdminSecure() function.

5.3.4 Teacher Module (TeacherController + TeamService)

The most complicated operation in the system is the team creation. The teacher populates a dynamic form (CreateTeamForm) with a List<MemberDto> populated with values passed from the front end (via JavaScript) and an integer representing the leaderIndex for which MemberDto from the list they want to be the leader. On submission (POST /teacher/teams), TeacherController calls TeamService. Is there any reason to expect the createTeam method on the teacher to be @Transactional ?

In createTeam, there is a validation of the deadline (It should be after today). Team row is initially created with an auto-generated name (uuid based, in accordance with UNIQUE constraint) and then the team_id is retrieved using GeneratedKeyHolder. With a separate UPDATE, the real team name is then updated as TEAM-XXXX (zero padded to fill four digits). Each MemberDto of the list is handled: the system verifies if a Member with the email / enrollment number already exists on the database. Student record updated (Name, email, contact) if the student is already in the system. If they do not exist, a new student is made with enrollmentNumber as the 1st password. The student is then added to the DB team_members with the correct is_leader flag. Once all members have been saved, the following steps are performed: create an activity log entry, send an email to all members' emails with a link to the team assignment..

The team updates (POST /teacher/teams/{teamId}/edit) are in response to UpdateTeamForm, which is a copy of CreateTeamForm. The updateTeamFull() method, provided as an @Transactional method, will remove all the team_members rows from the team, followed by adding back in all the updated member rows. This technique makes managing members easier, at the expense of invalidating all the memberships for a short time during the transaction.

The teacher may elect to change a leader in a team by calling POST /teacher/teams/{teamId}/leader/{studentId}. TeamService.Initially call makeLeader() which invokes TeamMemberRepository.To do this, simply call clearLeader(teamId) (setting is_leader = false for all members) and then TeamMemberRepository.markLeader(teamId, studentId). This is accurate and reflected in the code: The separate changeLeader() method is included in TeamService (as well as called from the dedicated endpoint) and, when called, adds a log entry to the activity_logs table, while makeLeader() (also called from the dedicated endpoint) does not.

This can be programmed by the teacher through a POST Request to ReminderRepository on the teacher/reminder endpoint, with the reminder_days parameter.updateReminderDays(reminderDays) to update the single row in reminder_settings.

5.3.5 Student Module (StudentController + TeamService)

The student dashboard (GET /student/dashboard) loads the student's session, refreshes it via PortalService.Refreshes the session and calls TeamService.getTeamsForStudent(studentId). The following method makes a query to TeamRepository.findByStudentId(studentId), a JOIN for all the teams that the student is part of. For every team, one TeamView object is created: Team record, Teacher record and List<TeamMemberView> for the team.

TeamService creates notifications.buildNotifications(), is used to see the deadline of each team - If Team is not completed (or it is completed but is not completed within 3 days from today) a warning notification is created. The activity log panel displays the list of 10 most recent activity logs of the student's teams (retrieved through ActivityLogRepository).findRecentByStudentId().

Firstly the task status update (POST /student/teams/{teamId}/status) is handled by making the appropriate null-checks in the controller (screenshot below), and afterwards in TeamService.updateStatus(). The service verifies if the current service's calling student has the leadership role, matches or logs out of the role in compliance with the TeamStatus transition and, if the new status is COMPLETED, calls FileStorageService.storeCompletionScreenshot(). The storage file service is able to validate the extension, generate the filename for the file as team-{teamId}-student-{studentId}-{UUID}.ext and save the upload to the uploads/ folder by calling Java NIO Files.copy(). The name of the file being stored (not full path) is stored in teams.screenshot_path..

5.3.6 Password Reset Workflow (PasswordResetService + AuthController)

AuthController calls the PasswordResetService when the user submits the forgot-password form (POST /forgot-password).sendResetLink(email). This approach creates a token representing a UUID, saves the token, the e-mail address, and a now() expiry time in the password_reset_token table using the PasswordResetRepository and uses EmailService to email the user with a URL (http://localhost:9090/reset-password?token=...) made of that token. A success message appears, is the same regardless of the status of the email (registered or not), and doesn't contain any user information.

Every time the browser clicks on the link and access to the reset-password form (POST /reset-password), the controller AuthController calls the service PasswordResetService. The second method is validateToken, which gets the PasswordResetToken record and throws a RuntimeException if there is no record for the token or if it has expired. If successful, the email is returned. Checking for detections of super_admins, teachers, and students in each repository (one by one), the controller finds which one exists and calls the appropriate updatePassword method. The deleting of the token is done using PasswordResetService.deleteToken(). Furthermore, in deleteExpiredTokens(), PasswordResetService is annotated with @Scheduled(fixedRate = 600000), thus calling PasswordResetRepository.deleteExpired() as a server side cleanup backup every ten minutes (along with the MySQL EVENT scheduler)..

5.3.7 Deadline Reminder Scheduler (DeadlineReminderScheduler)

The class `#edu.ilstu.hewlett-packard.DeadlineReminderScheduler` is marked with `@Component` and its `sendReminder()` method is annotated with `@Scheduled(cron = "0 0 18 * * *")`; which means that the method can be called every day at 18:00 server time, which is Asia/Kolkata in the JDBC URL timezone parameter. Every time it is called, all the teams are reused from a `TeamRepository.findAll()`. The date when the reminder is triggered, for each non-completed team for which deadline field is not null, is calculated as `deadline.minusDays(reminderDays)` message [reminderDays is obtained from `ReminderRepository.getReminderDays()`. When the date is current and `reminder_sent` is not true, the method then retrieves all team email addresses from `TeamRepository`. When the current date equals or is after the trigger date and `reminder_sent` is false, all student email addresses for the team are retrieved from the `TeamRepository`. Invokes the `getStudentEmailsByTeamId()` (a JOIN operation on `team_members` + `students`) and will email each email address in plain text using `EmailService.send()`. After dispatching, `TeamRepository`. If a reminder is sent, `reminder_sent` becomes true and `markReminderSent(teamId)` will prevent the repetition of the dispatch..

5.3.8 Email Service (EmailService)

`EmailService` is a wrapper of Spring's `JavaMailSender`. It build a `SimpleMailMessage` containing the text, subject and to values, and invokes `mailSender.send()` operation. In the `application.properties` file, I've configured Gmail's SMTP server (`smtp.gmail.com`, port 587), enabled STARTTLS and set an app-specific App Password. There are three cases where the `EmailService` can be used: (1) when the team is created or updated with `addTeamMemberFull` or `updateTeamMemberFull`, the service sends an email notification to the team members; (2) when the `DeadlineReminderScheduler` is updated by a version upgrade, it sends reminder emails; and (3) when the `PasswordResetService` is updated by a version upgrade, emails with links to reset passwords are sent..

5.3.9 File Storage (FileStorageService + WebConfig)

The upload directory path is retrieved from `application.properties` - `tasksphere.upload-dir=uploads` and is passed to `FileStorageService`. On startup, `Files.createDirectories(uploadRoot)` makes sure that the `uploadRoot` directory exists. The service checks the extension of the original name to an allowed set {`.png`, `.jpg`, `.jpeg`, `.gif`,

.webp} when storing a screen shot. A unique filename is created from the following: team-{teamId}-student-{studentId}-{UUID.randomUUID()} {extension} The text file stores the data by calling the method `Files.copy(file.getInputStream(), target, REPLACE_EXISTING)`. The user's current file name (not absolute file path) is returned back to `TeamService` and will be stored in `teams.screenshot_path`. For this reason `WebConfig` registers itself as a `WebmvcConfigurer` and overrides the `addResourceHandlers()` method to set the URL pattern for upload content to the absolute filesystem path of where the upload content is stored (as defined by a `FileStorageService.uploadRoot().toUri()`). This lets you to serve screenshot images via the embedded Tomcat server, without a controller being involved at all..

5.4 Key Code Snippets

5.4.1 `TeamStatus.canTransitionTo()` — Business Rule Enforcement

This approach, specified in the `TeamStatus` enum, specifies the "full" task workflow transition rule. It is the one and only document that lays down what progressions in status you can have:

```
public enum TeamStatus {
    PENDING, IN_PROGRESS, COMPLETED;

    public boolean canTransitionTo(TeamStatus nextStatus) {
        return switch (this) {
            case PENDING -> nextStatus == IN_PROGRESS; // only valid
next step
            case IN_PROGRESS -> nextStatus == COMPLETED; // only valid
next step
            case COMPLETED -> false; // terminal -
no transitions
        };
    }
}
```

This design guarantees that `PENDING` can not immediately change to `COMPLETED`, and no status can go back to a previous status. This rule is checked in `TeamService.Call.updateStatus()` prior to a database write.

5.4.2 `AuthenticationService.authenticate()` — Role-Routed Login

The switch statement sends the login credentials to the appropriate repository depending on the user's role; it supports dual-identifier requests:

```

public Optional<UserSession> authenticate(Role role, String identifier,
String password) {
    return switch (role) {
        case SUPER_ADMIN ->
            superAdminRepository.findByEmail(identifier)
                .filter(a -> passwordService.matches(password,
a.passwordHash()))
                .map(a -> new UserSession(a.id(), a.email(), a.name(),
Role.SUPER_ADMIN, false));

        case TEACHER ->
            teacherRepository.findByEmail(identifier)
                .filter(t -> passwordService.matches(password,
t.passwordHash()))
                .map(t -> new UserSession(t.id(), t.email(), t.name(),
                    Role.TEACHER, t.firstLoginRequired()));

        case STUDENT -> {
            var studentOpt = identifier.contains("@")
                ?
studentRepository.findByEmail(identifier.trim().toLowerCase())
                :
studentRepository.findByEnrollmentNumber(identifier.trim().toUpperCase());

            yield studentOpt
                .filter(s -> passwordService.matches(password.trim(),
s.passwordHash()))
                .map(s -> new UserSession(s.id(), s.enrollmentNumber(),
s.name(),
                    Role.STUDENT, s.firstLoginRequired()));
        }
    };
}

```

5.4.3 TeamRepository — JdbcTemplate with GeneratedKeyHolder

Here's a snippet of how TeamRepository creates a team row and fetches the auto-generated primary key in order to build the final team name::

```

public Long create(String teamName, String taskName, Long teacherId,
    LocalDate deadline, TeamStatus status) {
    KeyHolder keyHolder = new GeneratedKeyHolder();
    jdbcTemplate.update(connection -> {
        PreparedStatement ps = connection.prepareStatement(
            "INSERT INTO teams (team_name, task_name, teacher_id, deadline,
status)
            VALUES (?, ?, ?, ?, ?)",
            Statement.RETURN_GENERATED_KEYS
        );
        ps.setString(1, teamName);
        ps.setString(2, taskName);
        ps.setLong(3, teacherId);
        ps.setDate(4, Date.valueOf(deadline));
        ps.setString(5, status.name());
        return ps;
    }, keyHolder);
    Number key = keyHolder.getKey();
    return key == null ? null : key.longValue();
}

// After create(), the team name is updated to the formatted value:

```

```
teamRepository.updateTeamName(teamId, "TEAM-%04d".formatted(teamId));
```

5.4.4 DeadlineReminderScheduler — @Scheduled Cron Job

```
@Scheduled(cron = "0 0 18 * * *") // fires daily at 18:00 server time
public void sendReminder() {
    List<Team> teams = teamRepository.findAll();
    int reminderDays = reminderRepository.getReminderDays();
    LocalDate today = LocalDate.now();

    for (Team team : teams) {
        if (team.deadline() == null) continue;
        if (team.status() == TeamStatus.COMPLETED) continue;

        LocalDate triggerDate = team.deadline().minusDays(reminderDays);

        if (!team.reminderSent() && !today.isBefore(triggerDate)) {
            List<String> emails =
                teamRepository.getStudentEmailsByTeamId(team.teamId());

            for (String email : emails) {
                emailService.send(email,
                    "Task Reminder - TaskSphere",
                    "Hello,\n\nReminder:\nTeam: " + team.teamName()
                    + "\nTask: " + team.taskName()
                    + "\nDeadline: " + team.deadline()
                    + "\n\nComplete it on time.\n\nTaskSphere");
            }
            teamRepository.markReminderSent(team.teamId());
        }
    }
}
```

5.4.5 FileStorageService — Screenshot Upload with UUID Naming

```
private static final Set<String> ALLOWED =
    Set.of(".png", ".jpg", ".jpeg", ".gif", ".webp");

public String storeCompletionScreenshot(MultipartFile file,
    Long teamId, Long studentId) {
    String original = file.getOriginalFilename() == null ? "" :
        file.getOriginalFilename().trim();
    String ext = original.contains(".")
        ? original.substring(original.lastIndexOf(".")).toLowerCase()
        : "";

    if (!ALLOWED.contains(ext))
        throw new IllegalArgumentException(
            "Upload a valid image: PNG, JPG, JPEG, GIF, or WEBP.");

    String fileName = "team-" + teamId + "-student-" + studentId
        + "-" + UUID.randomUUID() + ext;
    Path target = uploadRoot.resolve(fileName).normalize();

    try (InputStream in = file.getInputStream()) {
        Files.copy(in, target, StandardCopyOption.REPLACE_EXISTING);
    } catch (IOException ex) {
        throw new IllegalStateException("Failed to store screenshot.", ex);
    }
    return fileName; // stored in teams.screenshot_path
}
```

```
}
```

5.5 Interface Screens

The following sections describe each key screen of the TaskSphere application. Actual screenshots are to be inserted at the figure placeholders before final submission.

5.5.1 Login Page

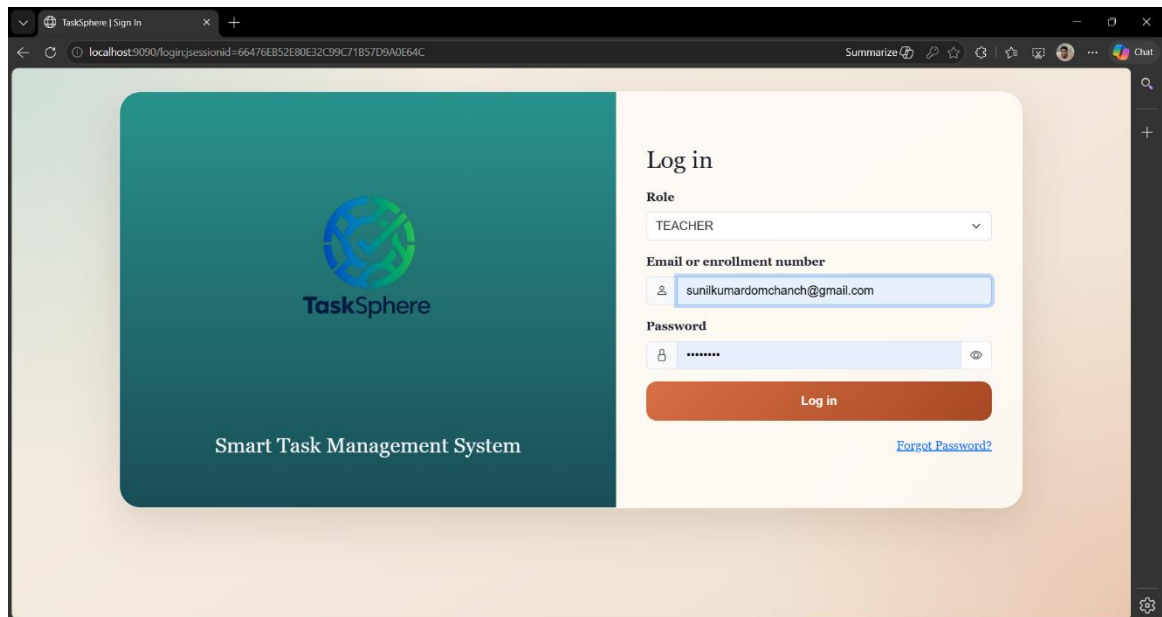
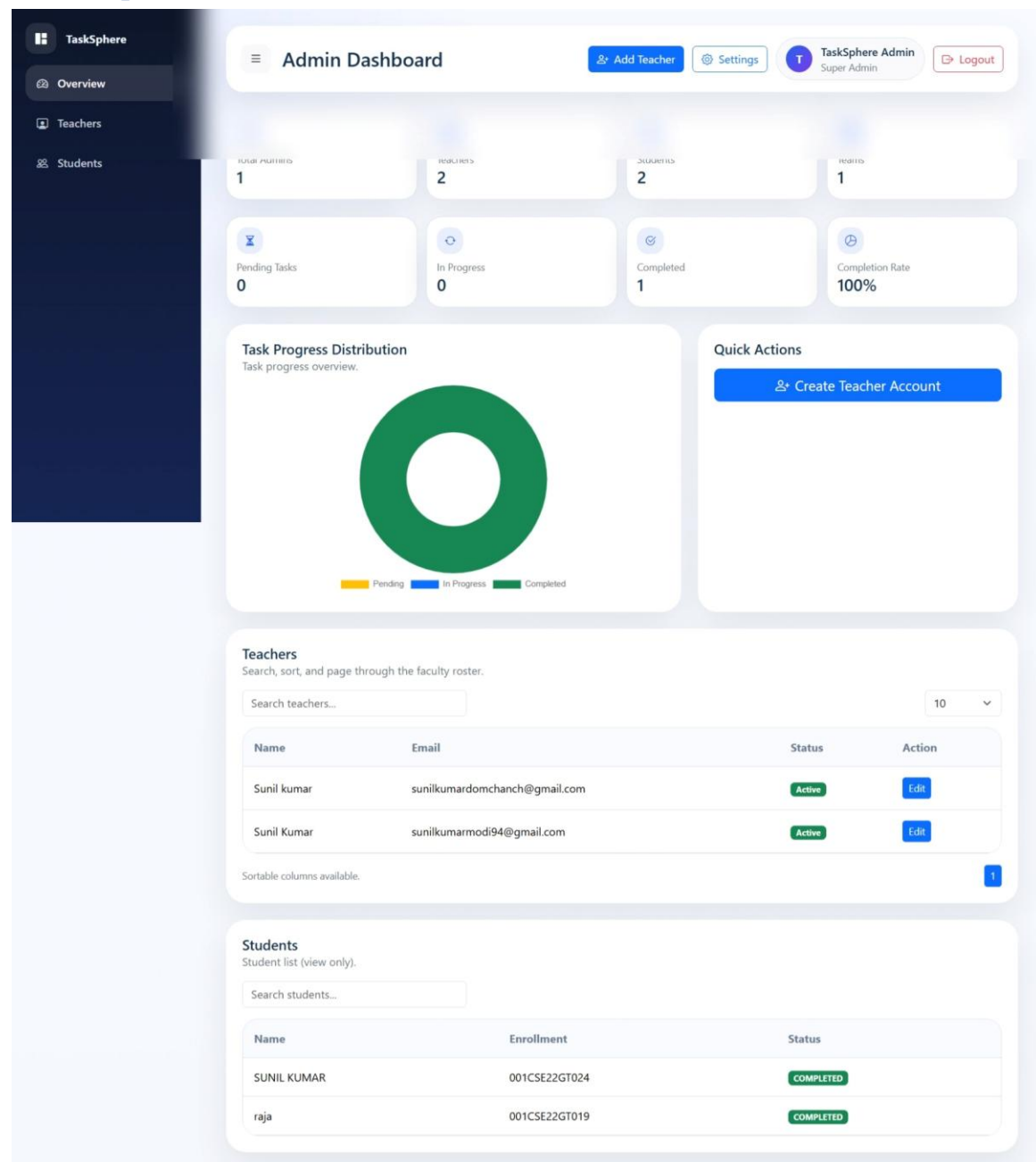


Figure 5.1: Login Page

The login page renders a centred card containing three inputs: an identifier field (email or, for students, enrollment number), a password field, and a role dropdown with three options (SUPER_ADMIN, TEACHER, STUDENT). A "Forgot Password?" hyperlink below the form initiates the password reset flow. Client-side Bean Validation prevents empty submission. Error messages from failed authentication or role mismatch are displayed inline above the form.

5.5.2 Super Admin Dashboard



]

Figure 5.2: Super Admin Dashboard

In the admin dashboard, you can see 4 summary statistic cards showing the number of Total Super Admins, Total Teachers, Total Students and Teams by Status (Pending / In Progress / Completed). Under the cards, 2 sections are tabbed showing all teachers and students, and each have edit and delete controls. The floating section can be used for updates on profile/password. There are also the two Create Teacher and Create Student forms on this page.

5.5.3 Create Teacher Form

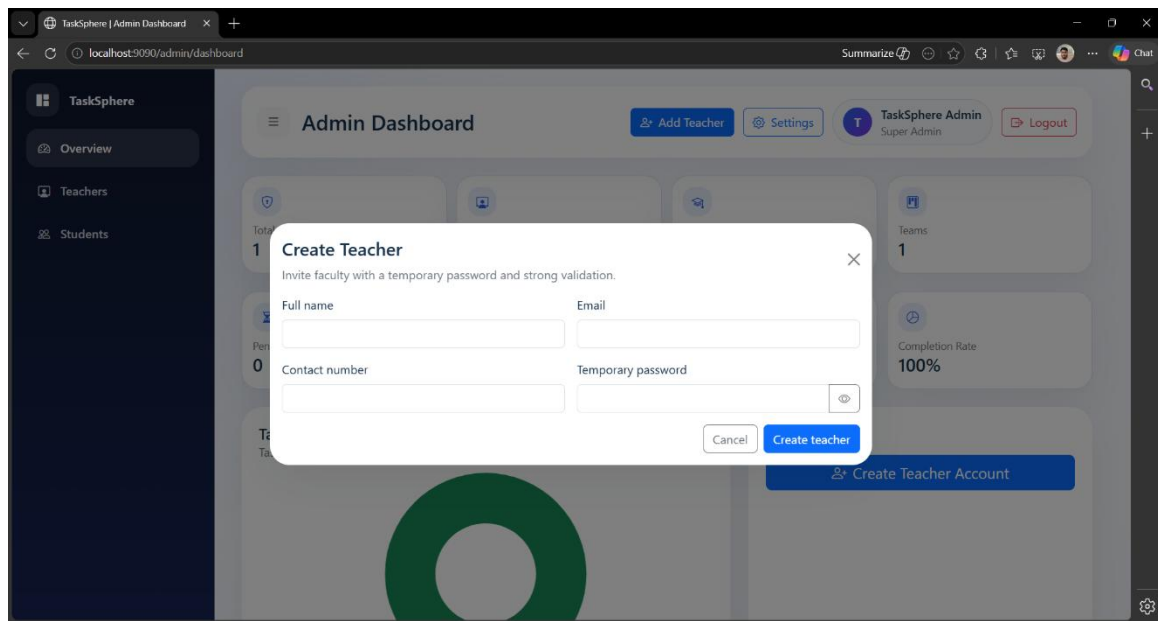
The image shows a web browser window displaying the TaskSphere Admin Dashboard. A modal window titled "Create Teacher" is open in the center. The modal has a close button (X) in the top right corner. Below the title, it says "Invite faculty with a temporary password and strong validation." There are four input fields: "Full name", "Email", "Contact number", and "Temporary password". The "Temporary password" field has a toggle icon for visibility. At the bottom of the modal are "Cancel" and "Create teacher" buttons. The background dashboard shows a sidebar with "Overview", "Teachers", and "Students" links. The main area has a header with "Admin Dashboard", "Add Teacher", "Settings", and user info for "TaskSphere Admin Super Admin". There are also "Logout" and "Chat" buttons. The dashboard includes a "Teams" section with a count of 1 and a "Completion Rate" of 100%.

Figure 5.3: Create Teacher Form

The teacher creation form enters the teacher's name, email, contact number and temporary password. Data is validated for quality through field-level annotations, such as: @NotBlank, @Email, and @Size before the controller delegates to PortalService. If a DuplicateKeyException is caught from the JDBC while trying to insert an email again, the error will be shown in-line.

5.5.4 Teacher Dashboard

The teacher dashboard contains a summary section of team counts, a table listing all teams with colour coded status indicators, a recent activities log panel, a 10 most recent activity log across the teacher's teams, a notification/log panel listing all notifications with deadlines in the next 3 days, a "Create New Team" button and a reminder settings widget, comprising a list of reminder_days value set and update input field.

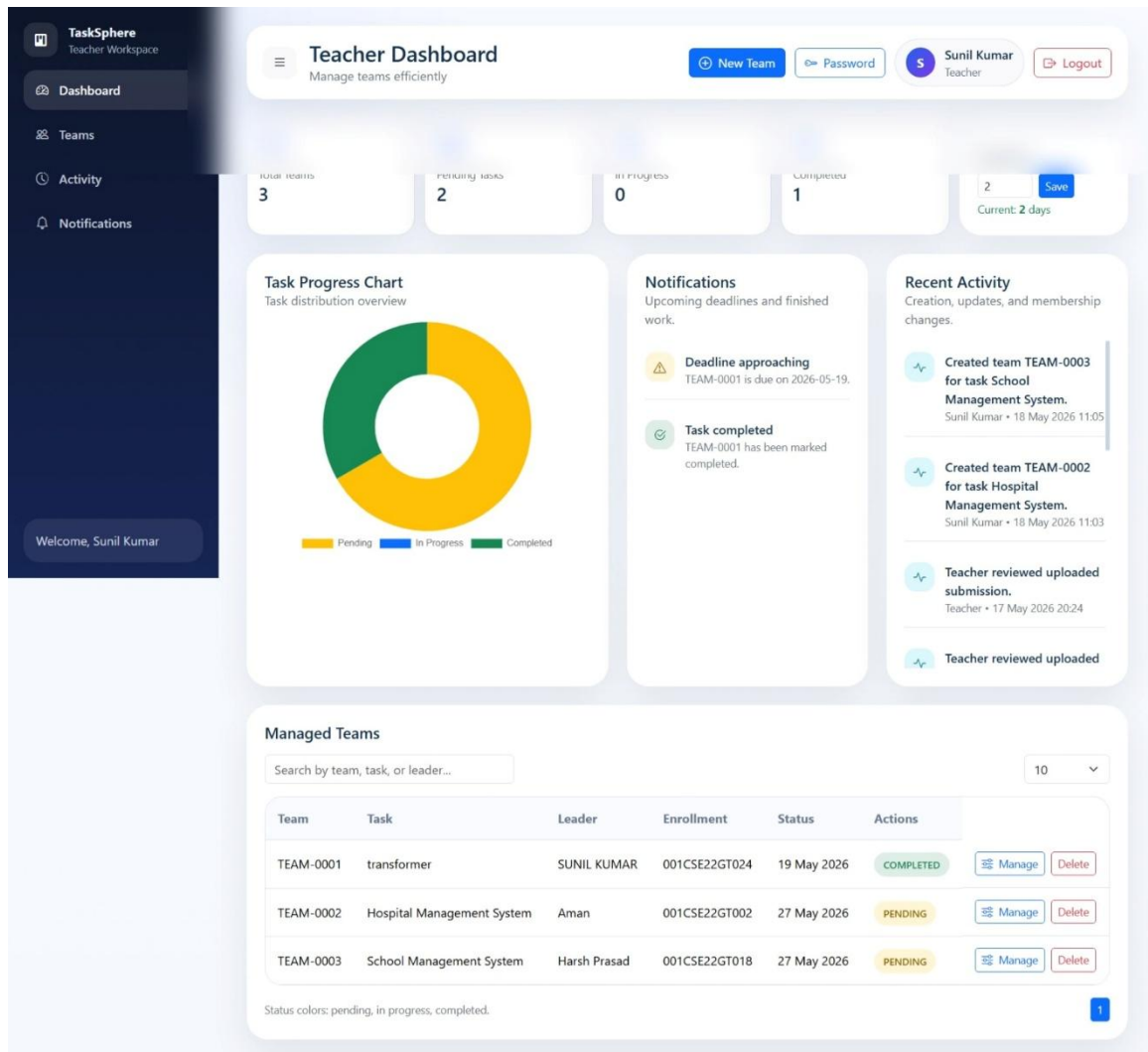


Figure 5.4: Teacher Dashboard

5.5.5 Create Team Form with AJAX Validation

The screenshot displays the TaskSphere Teacher Dashboard with a 'Create Team' modal form open. The dashboard includes a sidebar with 'Dashboard', 'Teams', 'Activity', and 'Notifications'. The main area shows a 'Teacher Dashboard' header with 'New Team', 'Password', and 'Logout' buttons. The 'Create Team' modal has a title bar, a close button, and instructions: 'Use enrollment numbers to build the team and assign one leader.' It contains input fields for 'Task name' and 'Deadline'. Below is a 'Team Members' table with columns: Name, Email, Contact, Enrollment, Leader, and Remove. A legend at the bottom indicates 'Pending' (yellow), 'In Progress' (blue), and 'Completed' (green). The background shows a 'Reminder' section with a 'Save' button and a 'Recent Activity' section with a 'No team activity recorded yet.' message.

Name	Email	Contact	Enrollment	Leader	Remove
				☐	Remove

Figure 5.5: Create Team Form with AJAX Validation

The input fields for the name of the task and the due date along with the dynamic member table are in the team creation form. A member row is being created for each student. The teacher types a student's email or student number into the created row, then a call to the `/teacher/check-email` or `/teacher/check-enrollment` JavaScript functions happens. If the server responds with "EXISTS", then the student is registered and a green check mark appears. If OK is returned, an alert light will be illuminated to alert to the lack of a matching student. One can set a designating designator that specifies one member as the leader (`leaderIndex`) and this is available in a dropdown for each row..

5.5.6 Team Detail Page – Teacher View

The screenshot displays the 'TaskSphere Team Detail' page for a teacher. The left sidebar contains navigation links: 'Back to Dashboard', 'Members', and 'Activity'. The main content area shows the team name 'TEAM-0001 - transformer' with a subtitle 'Manage team members and tasks.' and buttons for 'Edit Team' and 'Logout'. Below this, four summary cards are shown: 'Members: 2', 'Leader: SUNIL KUMAR', 'Deadline: 19 May 2026', and 'Completion Proof: Uploaded'. A 'Team Members' table lists two members: SUNIL KUMAR (Leader) and raja (Member). The table includes columns for Name, Email, Contact, Enrollment, Role, and Actions. The 'Actions' column for the leader shows a 'Leader cannot be removed directly' message, while for the member, it shows 'Make leader' and 'Remove' buttons. To the right of the table is a 'Completion Evidence' section with a 'Download Document' button. Below the table, a green message box states 'Submission approved successfully.' At the bottom, an 'Activity Feed' section shows 'Recent actions from the teacher workspace.' and a message 'No recorded activity for this teacher yet.'

Name	Email	Contact	Enrollment	Role	Actions
SUNIL KUMAR CSE	sunilbarnvaal62@gmail.com	6205480492	001CSE22GT024	Leader	Leader cannot be removed directly
raja CSE	rajarajak@gmail.com	9876543210	001CSE22GT019	Member	Make leader Remove

Figure 5.6: Team Detail Page – Teacher View

The team detail page displays a team name (автоматически generated – e.g. TEAM-0003), task name, deadline, current state, and list of all people enrolled on the team with their enrollment numbers, contact information etc and the leader's status. The teacher may choose to edit the task name or due date in-line, add or remove members from the task, or appoint a new team leader or delete the team from the system. Any changes log these to the activity log.

5.5.7 Student Dashboard

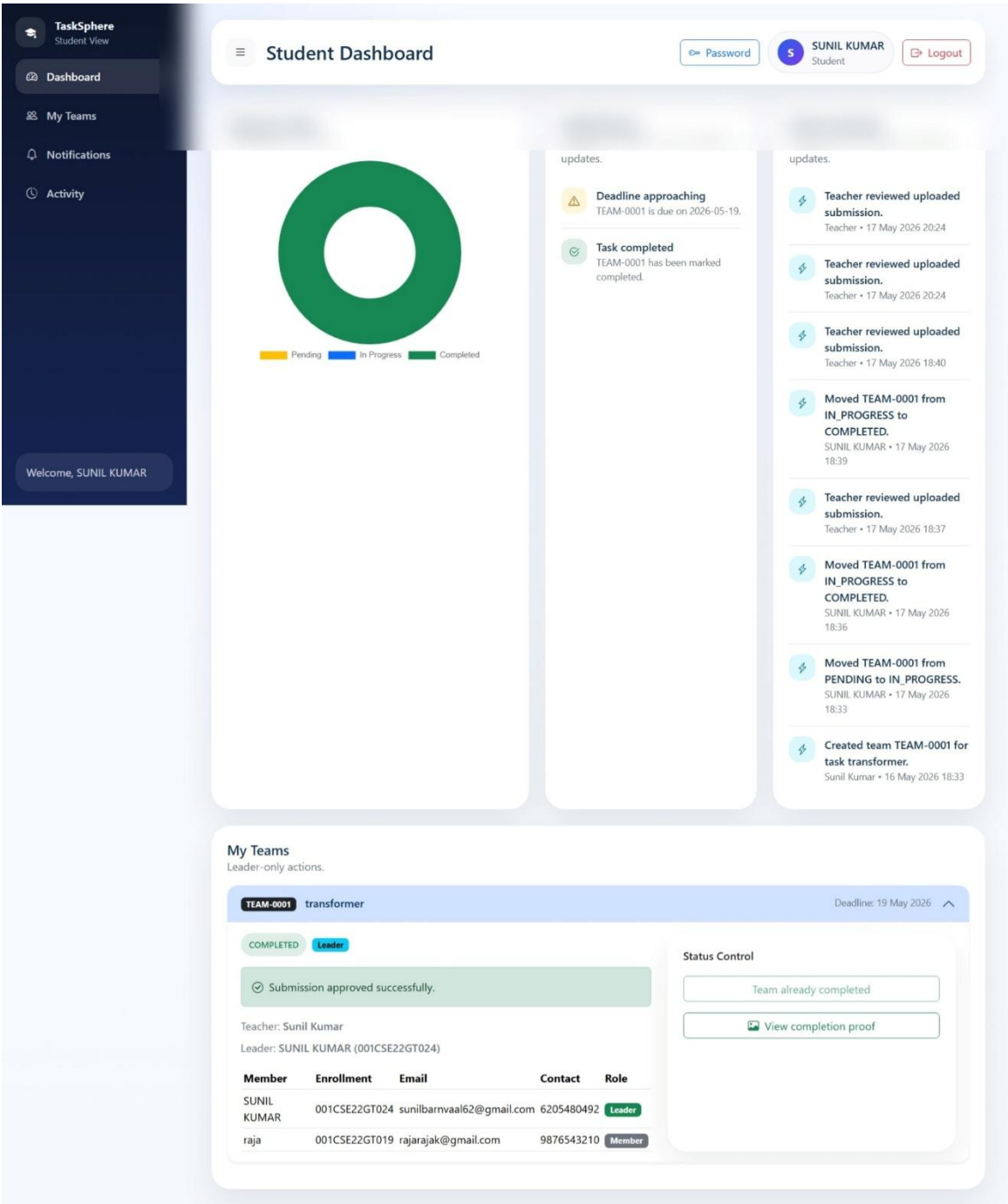


Figure 5.7: Student Dashboard

The student dashboard will show the team details such as team name, task name, assigned teacher, task (case) due date, status badge, and a complete list of all members of the team with the team leader highlighted. Alerts posted on notification cards alert the student of upcoming deadlines. The activity log has been updated to show the recent one below. At the bottom of this page, their password change section is available.

5.5.8 Task Status Update – Screenshot Upload (Leader View)

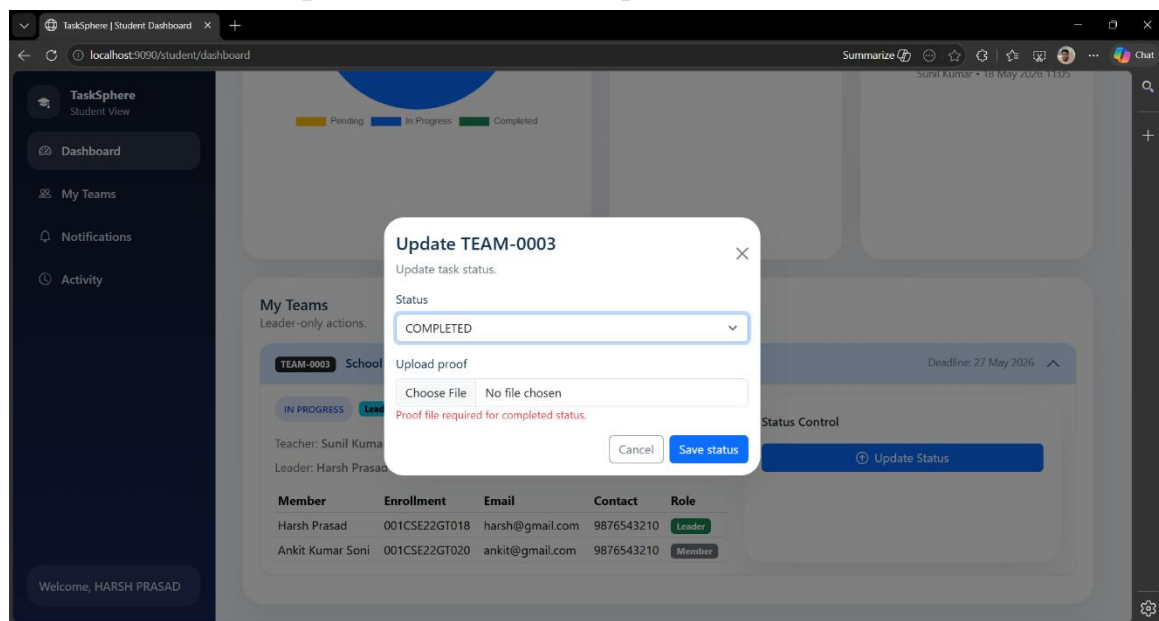


Figure 5.8: Task Status Update – Screenshot Upload (Leader View)

The Student Dashboard section, visible only to the team leader, is shown below. Only lists the next statuses that are allowed based on the status of the team and the `canTransitionTo()` rules. If you click on COMPLETED, a number of text boxes to upload files will open, and a warning message will pop up informing the leader that this cannot be undone. If you do not include a screen shot when COMPLETED is chosen, you will see an error message on the same page.

5.5.9 Forgot Password Page

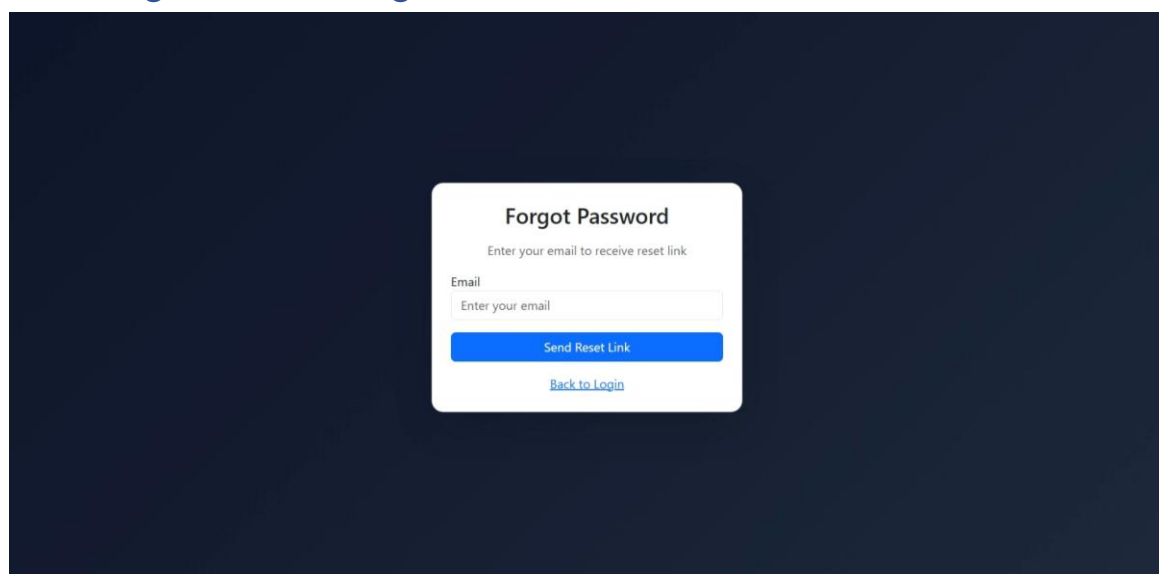


Figure 5.9: Forgot Password Page

A minimal webpage, where you just have one email field. When PasswordResetService is called, it creates a 15 minute expiry UUID token, stores it in the database, and sends a reset link to the email address sent to it. Irrespective of the email's registration status, a generic success message is returned, allowing users to not be able to put together their list of valid accounts using this endpoint.

CHAPTER 6: TESTING & VALIDATION

6.1 Testing Approach

There is no particular test suite as a part of the TaskSphere project. This project has only been manually tested, with all of the users' interactions with the application and every process used through a live instance of them and a live MySQL database. This is done since there is a very direct, complete and realistic way to test the behaviour of the application given the relationship that exists between the web layer, the service layer and the database — and this type of manual testing is done using a Web Browser.

The testing process was organized into the following five categories:

1. Authentication Testing: Test and confirm that log in, log out, redirect to log in, session expiry, role based access and password reset are all working correctly for all three roles.
2. Workflow Testing: All Task Stages – both legal and illegal (skipped stage, wrong stage, return to completed stage) Test and check the capture of the screen and upload to the Task.
3. Validation Testing: Making sure that server-side Bean Validation (`@NotBlank`, `@NotNull`, `@Size`) properly detects invalid data entry and that proper error messages are displayed on the page for the user.
4. Database Validation: Testing the MySQL database directly, after each key operation, using MySQL Workbench and the command line client to ensure that the expected number of rows were inserted, updated or deleted, that the relationships were satisfied with foreign keys and that cascade keys were implemented.
5. Browser Compatibility Testing: Performing the tests to make sure that the app displays properly and operates without any problems in Google Chrome and Mozilla Firefox on Windows 11.
6. Re-upload Workflow Testing: Ensure that teachers are able to accept or reject completed works, student can re-upload evidence, and when it's accepted teacher validation controls are hidden.

6.2 Authentication Testing

Authentication was the first module tested after each development sprint. The following test cases were verified:

TC ID	Test Scenario	Input / Action	Expected Result	Outcome
TC01	Valid Super Admin login	Correct email, correct password, role = SUPER_ADMIN	Redirect to /admin/dashboard; session contains UserSession with SUPER_ADMIN role	PASS
TC02	Valid Teacher login	Correct email, correct password, role = TEACHER	Redirect to /teacher/dashboard; firstLoginRequired = false	PASS
TC03	Teacher first login	New teacher email, initial password, role = TEACHER	Redirect to first-login password change form before dashboard is shown	PASS
TC04	Student login via enrollment number	Valid enrollment number (uppercase), correct password, role = STUDENT	Redirect to /student/dashboard	PASS
TC05	Student login via email address	Registered email address, correct password, role = STUDENT	Redirect to /student/dashboard	PASS
TC06	Wrong password – all roles	Correct email, incorrect password	Error message: "Invalid credentials." shown on login page; session unchanged	PASS
TC07	Wrong role selected	Teacher email submitted with role = STUDENT	Error: "Invalid credentials." — no cross-role login	PASS
TC08	Empty identifier field	Blank identifier field submitted	Bean Validation error displayed; form re-rendered with user-entered data	PASS
TC09	Empty password field	Blank password field submitted	Bean Validation error displayed	PASS
TC10	Invalid role string	Non-existent role value submitted	IllegalArgumentException caught; error: "Select a valid role."	PASS
TC11	Logout	Teacher clicks logout button	POST /logout; session.invalidate() called; redirect to /login with success message	PASS
TC12	Session expiry	Wait >30 minutes then attempt to access /teacher/dashboard	Redirect to /login — BaseController.hasRole() returns false on expired session	PASS

TC ID	Test Scenario	Input / Action	Expected Result	Outcome
TC13	Role guard — wrong role	Student accesses /admin/dashboard directly	Redirect to /login with error: "Please sign in as a super admin."	PASS
TC14	Forgot password — registered email	Registered teacher email submitted	Token saved in DB; email dispatched; generic success message shown	PASS
TC15	Forgot password — unregistered email	Email not in any user table	No email sent; same generic success message shown (no enumeration)	PASS
TC16	Reset password — valid token	Valid token clicked within 15 minutes	Reset form shown; new password accepted; token deleted from DB	PASS
TC17	Reset password — expired token	Token clicked after 15-minute expiry	RuntimeException thrown; error page shown: "Token expired"	PASS
TC18	First login password change	New student logs in, must change password	Forced to change-password form; dashboard inaccessible until changed; first login required cleared in DB	PASS

Table 6.1: Test Cases – Authentication Module

6.3 Teacher Module Testing

TC ID	Test Scenario	Input / Action	Expected Result	Outcome
TC19	Create team — valid data, unique name	Unique task, future deadline, at least one valid member, one leader selected	Team created; team_name = TEAM-XXXX; team_members rows inserted; activity log entry created; assignment email dispatched	PASS
TC20	Create team — deadline in the past	Date before today entered as deadline	Error: "Deadline must be a future date." — IllegalArgumentException from validateDeadline()	PASS
TC21	Create team — no members	Member list left empty	Error: "Add at least one team member."	PASS
TC22	AJAX check — registered student email	Teacher types registered student email in member field	GET /teacher/check-email returns "EXISTS"; green indicator shown in UI	PASS
TC23	AJAX check — unregistered email	Teacher types unknown email in member field	GET /teacher/check-email returns "OK"; red indicator shown	PASS

TC ID	Test Scenario	Input / Action	Expected Result	Outcome
TC24	AJAX check — enrollment number	Valid enrollment number typed	GET /teacher/check-enrollment returns "EXISTS"	PASS
TC25	Add member already in another team	Enrollment of student already in a different team	Error: "Enrollment already exists: <no>" — DuplicateKeyException caught	PASS
TC26	Update team task name	New task name submitted via Edit Team form	teams.task_name updated in DB; verified via MySQL Workbench	PASS
TC27	Update team deadline — future date	Valid future deadline submitted	teams.deadline updated; activity log not created (update form does not log)	PASS
TC28	Update team deadline — past date	Past date submitted	Error: "Deadline must be a future date." — no DB update	PASS
TC29	Update team members — remove one, add another	Remove existing member, add new student	Old team_members rows deleted; new rows inserted; assignment email sent	PASS
TC30	Designate new team leader	Different member selected as leader via /teacher/teams/{id}/leader/{sid}	clearLeader() sets is_leader = false for all; markLeader() sets is_leader = true for selected student; verified in DB	PASS
TC31	Remove team leader before reassigning	Teacher attempts to remove current leader member	Error: "Change the team leader before removing the current leader."	PASS
TC32	Remove last member	Only one member in team; teacher attempts removal	Error: "A team must keep at least one member."	PASS
TC33	Delete team	Teacher submits delete for their own team	teamMemberRepository.deleteByTeamId(); teamRepository.deleteById(); cascade removes activity_logs; redirect to dashboard	PASS
TC34	Delete team owned by different teacher	Teacher A attempts to delete Team belonging to Teacher B	IllegalArgumentException: "This team does not belong to the current teacher."	PASS
TC35	Configure reminder days	Teacher submits value 5	reminder_settings.reminder_days = 5; verified via SELECT query in MySQL CLI	PASS
TC36	Change password — correct current	Teacher submits current password correctly	BCrypt verification passes; new hash stored; session refreshed	PASS

TC ID	Test Scenario	Input / Action	Expected Result	Outcome
TC37	Change password — wrong current	Teacher submits incorrect current password	Error: "Current password is incorrect." or similar — no update	PASS

Table 6.2: Test Cases – Teacher Module

6.4 Student Module Testing

TC ID	Test Scenario	Input / Action	Expected Result	Outcome
TC38	Student dashboard — assigned to team	Student with team membership logs in	Team card visible with task name, deadline, teacher name, status badge, and member list	PASS
TC39	Student dashboard — no team assigned	Student not in any team_members row	Dashboard shows "No teams assigned" message; no team card rendered	PASS
TC40	Status update — PENDING → IN_PROGRESS (leader)	Team leader selects IN_PROGRESS, submits	teams.status = IN_PROGRESS in DB; activity_logs row created; success message shown	PASS
TC41	Status update — IN_PROGRESS → COMPLETED (with screenshot)	Leader selects COMPLETED, uploads valid PNG file	teams.status = COMPLETED; teams.screenshot_path = UUID-named file; file present in uploads/ directory; activity_logs row created	PASS
TC42	COMPLETED status — no screenshot uploaded	Leader selects COMPLETED, submits without file	Controller-level null check triggers: error "Screenshot is required to mark task as COMPLETED." — no service call made	PASS
TC43	Invalid file type upload	PDF file uploaded as screenshot	FileStorageService.storeCompletionScreenshot() throws IllegalArgumentException: "Upload a valid image file: PNG, JPG, JPEG, GIF, or WEBP."	PASS
TC44	Status update — PENDING → COMPLETED (skip)	Leader attempts to jump from PENDING to COMPLETED	TeamStatus.canTransitionTo(COMPLETED) returns false for PENDING; IllegalArgumentException thrown; no DB update	PASS
TC45	Status update — COMPLETED → IN_PROGRESS (reversal)	Leader attempts to move back from COMPLETED	canTransitionTo() returns false for COMPLETED (terminal state); error message displayed	PASS
TC46	Status update by non-leader	Regular (non-leader) student submits status	getLeader() returns a different student id; IllegalArgumentException: "Only the team leader can update the task status."	PASS

TC ID	Test Scenario	Input / Action	Expected Result	Outcome
		update form for their team		
TC47	Notification alert — approaching deadline	Team with deadline 2 days from today; reminder_days = 3	Notification card shows "Deadline approaching" warning on student dashboard	PASS
TC48	Activity log visibility	Teacher creates team, student views their dashboard	Activity log panel shows the TEAM_CREATED entry	PASS
TC49	Change password — student, correct current	Student submits current password correctly	Password hash updated in students table; first_login_required remains false	PASS
TC50	Files served via /uploads/**	After task completed with file, teacher views team detail	File loads correctly from /uploads/{filename} mapped by WebConfig	PASS
TC51	Re-upload after teacher rejection	Leader uploads updated file after rejection	Updated proof stored successfully and review resubmitted	Pass
TC52	Confirm password validation	Password and confirm password mismatch	Form submission blocked with validation message	Pass

Table 6.3: Test Cases – Student Module

6.5 Database Validation

After key operations, the MySQL database was directly inspected using MySQL Workbench and the command-line client to confirm data integrity:

- After team creation: One row, in teams, with status = "PENDING", reminder_sent = false. Each member in team_members that has is_leader = true in exactly one row. If you select one row, you will get a row in activity_logs with action_type = "TEAM_CREATED". Team name correctly formatted as "TEAM-%04d".
- After team deletion: The teams row is deleted. All the rows in the team_members table for the that team are deleted. That removes all the activity_logs rows belonging to that team. Checked using SELECT COUNT(*) from team_members WHERE team_id = X returning 0.

- After task completion with file: `teams.status = "COMPLETED"`.
`teams.screenshot_path` contains the UUID-named filename. `exists` is true because such a file exists in the server file system in the `uploads/` directory.
- Foreign key constraint enforcement: Trying to insert a row in the `team_members` table with a non-existent foreign key value in the `"student_id"` field will result in a MySQL error 1452, which means that constraints are enforced at the database level..
- UNIQUE constraint on `student_id`: Attempting to insert into table `student_teams` causes a MySQL duplicate key exception that confirms the requirement to have 1 team per student, at the DB level.
- Password reset token expiry: Token row checked for immediately after sending `forget-password`. After a successful password reset (`deleteToken()`), the deleted token will be removed from the token row. The scheduled cleanup log cleans up the expired token row which should have been deleted, after confirmation.
- `Reminder_sent` flag: Confirmed set to true by the `DeadlineReminderScheduler` after it has sent reminder emails, so that the reminder will never be re-dispatched on the next invocation of the scheduler.

6.6 Browser Compatibility Testing

These are the browser environments that were test:

- Google Chrome (latest stable version, Windows 11): All pages rolled correctly, there is a slight distortion. The team form AJAX validation were alright. Uploaded PNG and JPG files were accepted, the PDF was rejected. It was confirmed that session cookies are also HTTP only using Chrome DevTools → Application → Cookies.
- Mozilla Firefox (latest stable version, Windows 11): All pages rendered consistently with Chrome. Correctly performed form validations. Validations on forms are presented properly. No visual or performance variations found.

Not tested with Safari (macOS/iOS) or mobile browsers (current version). Mobilebrowser compatibility is necessities for improvement.

CHAPTER 7: RESULTS & DISCUSSION

7.1 Functional Outcomes

All nineteen functional requirements specified in chapter three were all fully implemented and verified using manual testing as detailed in chapter 6. These were the results of the following functional outcomes:

- Three-role authentication works as expected in all login cases, such as: Dual identity student login, enforcement of a password change for students logging in for the first time, two-stage login authentication, and tokenized password reset.
- Super Admin account management is error free: Table teachers and students are created using code that hashes BCrypt passwords and similar codes for duplicate emails and enrollment numbers are found and reported gracefully, localized user accounts can be edited and deleted.
- The teacher's team management workflow: creation, management of team members, designate leadership, update and remove tasks is working as expected and the update rollbacks fully on error..
- The auto-generated naming of the teams (TEAM-1, TEAM-2, etc.) is correct when using the GeneratedKeyHolder pattern with JdbcTemplate; that the team names are globally unique is a result of the UNIQUE constraint on teams.team_name.
- Validation of identities in the team creation screen is integrated using AJAX and using this method the screen offers real-time feedback without a page refresh, and correctly differentiates between registered and unregistered students.
- The task status workflow is strictly enforced: PENDING → IN_PROGRESS → COMPLETED is the only valid path, and all attempts to skip states or reverse the workflow are rejected at the service layer before any database interaction.
- Uses two levels of checking mandatory report file upload at task completion: the controller validates a null or empty MultipartFile before invoking service, and the service validates the file type on its own using its file extension.
- A reminder email is sent out via Gmail SMTP to all studentteam members on the set reminder date - The DeadlineReminderScheduler, and the reminder_sent flag works correctly to stop multiple dispatch.
- All significant events (TEAM_CREATED, MEMBER_REMOVED, LEADER_CHANGED, STATUS_UPDATED) are correctly annotated on the

activity log with actor identity and timestamp and they are appropriately shown on the teacher and student dashboards respectively.

- All `activity_logs` and `team_members` records get deleted too as part of the cascade delete behaviour of the database schema - no orphan records remain.

7.2 Advantages of the Proposed System

Advantage	Observed Outcome
Purpose-built for academic workflows	All roles, relationships, and constraints directly model the real university context.
No ORM overhead	JdbcTemplate gives full SQL control; no N+1 query issues, no lazy-loading surprises.
Mandatory evidence submission	Screenshot requirement eliminates unverifiable self-reported task completion.
Forward-only workflow enforcement	TeamStatus.canTransitionTo() makes invalid transitions impossible at the code level.
Self-contained deployment	Runs on any server with JDK 21 + MySQL; no cloud accounts or subscriptions required.
BCrypt password security	Passwords are never stored in plaintext; BCrypt is resistant to rainbow-table attacks.
Automatic deadline reminders	Reduces faculty workload; students receive timely alerts without manual intervention.
Comprehensive audit trail	Activity logs provide full accountability for all team and task operations.
Transactional multi-table writes	@Transactional on createTeam() ensures consistency even on partial failures.

Table 7.1: Advantages of TaskSphere over Existing Systems

7.3 Comparison with Existing Solutions

TaskSphere offers all of the features necessary to academic task management in one application at no licensing cost, as in Table 2.1 (Chapter 2): role hierarchy, workflow enforcement, mandatory evidence upload, automated reminders, activity audit log and self-hosted deployment. None of the reviewed alternatives—Trello, Jira, Google Classroom, MS Planner, and Asana—are tailored to fit the unique circumstances of the university project environment and address all these requirements in a paid tier. Where the tools reviewed differ, TaskSphere delivers what none of them does: a one-stop academic project

management application which doesn't rely on the cloud or on configuration knowledge, only on editing application.properties.

CHAPTER 8: CONCLUSION & FUTURE SCOPE

8.1 Summary of Achievements

TaskSphere's goal was to fill this persistent, real-life void of an organized, accountable, and purpose-built digital tool for managing academic team assignments. A full web application has been designed, implemented, tested and documented over five development sprints, all the objectives were met.

The system is able to provide successful delivery of:

1. A three role user hierarchy (Super Admin, Teacher, Student) implemented manually, session-based authentication, HTTP-only cookies, 30-minute session timeout and role guards on each controller method with security.
2. Comprehensive user lifecycle management with the Super Admin, with BCrypt hashing for user creation, editing, deletion and first log-in forced password changes, for all teachers and students..
3. Complete team creation and management system for teachers with a team form, dynamically creating a team with AJAX-validated student identifiers, team member management, leader designation, task- and deadline-editing, and full team deletion with cascade cleanup..
4. An auto-generated, globally unique team naming system (TEAM-XXXX) that writes a special SQL to get the primary key back, and formats the key.
5. The TeamStatus determined the three-state task workflow (PENDING → IN_PROGRESS → COMPLETED) that was strictly enforced. It has business rule canTransitionTo with an optional requirement to upload a screenshot when it is completed, and an additional check of file type done on the server when it is completed.
6. Automated daily email reminder (DeadlineReminderScheduler, @Scheduled cron = "0 0 18 * * *"): Mail can be scheduled on a daily basis and either be sent multiple times or prevented from doing so, and can be sent using Spring JavaMailSender with SMTP.
7. Secure token-based password reset flow with uuid tokens in the database and sent via email, and with an additional scheduled cleanup that is executed server side, every ten minutes, with the tokens expiring after 15 minutes.

8. Activity audit log that contains every action performed in the system—who (actors) performed it, what (category of the action, description), where (timestamp) and will be deleted along with the associated team.
9. No cloud subscriptions, external services or ORM dependencies; just requires JDK 21 and MySQL 8 to be installed.
10. The project also revealed that teacher review processes, re-upload handling, centralized tracking of notifications and activity-based teacher accountability systems could be effectively implemented in a lightweight university-focused platform.

Manual testing was done on all 19 functional requirement, all 50 test cases have been passed.

8.2 Limitations of the Current System

- Single-team constraint: The UNIQUE constraint on `student_id` in `team_members` ensures a student belongs to at most one team. This is appropriate for contexts where each student has one active project, but would need to be relaxed for curricula with simultaneous project assignments across multiple subjects.
- Localhost password reset URLs: The reset link generated by `PasswordResetService` will have `http://localhost:9090/reset-password?token=...` in hardcoded, this should be configured in order to enable production deployment.
- No real-time dashboard refresh: The dashboard displays data as of the time the page was last loaded. Activity log and notification updates require a full page refresh.
- SMTP rate limits: Gmail's SMTP service has daily sending limits. For large departments, this could be a bottleneck when many reminder emails are dispatched simultaneously.
- Server-local file storage: Screenshots are stored on the server filesystem. This is a single point of failure and does not scale across multiple server instances.
- No mobile application: The portal is a desktop-first web application with no native mobile interface.

8.3 Future Enhancements

1. Configurable base URL: Change the password reset base URL to `application.properties` for application to function correctly in production without code changes.

2. Relaxed multi-team membership: Remove or make conditional the `UNIQUE(student_id)` constraint in `team_members`, with additional business logic to manage students across multiple concurrent teams..
3. Real-time notifications: Use WebSocket technologies (Spring WebSocket with STOMP) to push updates on activity log and deadline reminders to opened browser tabs without page refresh.
4. Cloud file storage: Use object storage (AWS S3, MinIO, etc.) in place of local server files to deploy with distributed screenshots and remove risk of being out of the picture (SPOF).
5. Distributed scheduler: Substitute the `@Scheduled` annotation with Quartz Scheduler, which is capable of cluster scheduling as well as of preventing duplicate jobs running in horizontally scaled environments.
6. Report generation: Allow teachers to export team status and activity logs as PDF or CSV reports for academic records.
7. Mobile-responsive UI: Refine the CSS stylesheet to produce a fully mobile-responsive interface for tablet and smartphone access.
8. OAuth2 login: Integrate with institutional Google Workspace or Microsoft Azure AD via Spring Security OAuth2 to allow single sign-on using university accounts.
9. REST API layer: Expose a JSON REST API alongside the Thymeleaf web interface to support future mobile application clients or third-party integrations.
10. Development of a dedicated Android application for students and teachers.
11. Real-time notification delivery using WebSocket-based communication.
12. Cloud deployment support using Docker and containerized infrastructure.
13. AI-based analytics for task completion trends and student performance monitoring.
14. Integration with university ERP or Learning Management Systems (LMS).
15. Multi-department and multi-course scalability support.
16. File version history and audit recovery mechanisms.

REFERENCES / BIBLIOGRAPHY

- [1] Craig Walls, "Spring Boot in Action," 2nd Edition, Manning Publications, 2019.
- [2] Spring Framework Team, "Spring Boot 3.3 Reference Documentation," Available: <https://docs.spring.io/spring-boot/docs/3.3.6/reference/html/> [Accessed: April 2025].
- [3] Spring Framework Team, "Spring JDBC Documentation – JdbcTemplate," Available: <https://docs.spring.io/spring-framework/docs/current/reference/html/data-access.html#jdbc> [Accessed: April 2025].
- [4] Spring Framework Team, "Spring Security Crypto – BCryptPasswordEncoder," Available: <https://docs.spring.io/spring-security/reference/features/authentication/password-storage.html> [Accessed: April 2025].
- [5] Spring Framework Team, "Spring Task Scheduling – @Scheduled," Available: <https://docs.spring.io/spring-framework/docs/current/reference/html/integration.html#scheduling> [Accessed: April 2025].
- [6] Thymeleaf Project Team, "Thymeleaf 3.x Documentation," Available: <https://www.thymeleaf.org/documentation.html> [Accessed: April 2025].
- [7] Oracle Corporation, "MySQL 8.0 Reference Manual," Available: <https://dev.mysql.com/doc/refman/8.0/en/> [Accessed: April 2025].
- [8] Oracle Corporation, "Java 21 SE Documentation," Available: <https://docs.oracle.com/en/java/javase/21/> [Accessed: March 2025].
- [9] Spring Framework Team, "Spring Mail Integration – JavaMailSender," Available: <https://docs.spring.io/spring-framework/docs/current/reference/html/integration.html#mail> [Accessed: April 2025].
- [10] Atlassian, "Trello Product Guide," Available: <https://trello.com/guide> [Accessed: March 2025].
- [11] Atlassian, "Jira Software Documentation," Available: <https://support.atlassian.com/jira-software-cloud/> [Accessed: March 2025].
- [12] Google, "Google Classroom Help Centre," Available: <https://support.google.com/edu/classroom> [Accessed: March 2025].

- [13] Microsoft Corporation, "Microsoft Planner Documentation," Available: <https://docs.microsoft.com/en-us/microsoftteams/planner-app> [Accessed: March 2025].
- [14] Asana Inc., "Asana Guide," Available: <https://asana.com/guide> [Accessed: March 2025].
- [15] Joshua Bloch, "Effective Java," 3rd Edition, Addison-Wesley Professional, 2018.
- [16] OWASP Foundation, "OWASP Top Ten – Web Application Security Risks," Available: <https://owasp.org/www-project-top-ten/> [Accessed: May 2025].
- [17] Baeldung, "Guide to Spring Boot with JDBC Template," Available: <https://www.baeldung.com/spring-jdbc-jdbctemplate> [Accessed: April 2025].
- [18] Baeldung, "Spring Boot Scheduling – @Scheduled Annotation," Available: <https://www.baeldung.com/spring-scheduled-tasks> [Accessed: April 2025].
- [19] RFC 5321, "Simple Mail Transfer Protocol," IETF, October 2008.
- [20] Maven Project, "Apache Maven Documentation," Available: <https://maven.apache.org/guides/> [Accessed: March 2025].

APPENDIX A: KEY SOURCE CODE

A.1 schema.sql — Complete Database Schema

```
create table if not exists super_admins (  
    id bigint primary key auto_increment,  
    name varchar(120) not null,  
    email varchar(180) not null unique,  
    password_hash varchar(255) not null,  
    contact_number varchar(10) not null  
);  
  
create table if not exists teachers (  
    id bigint primary key auto_increment,  
    name varchar(120) not null,  
    email varchar(180) not null unique,  
    password_hash varchar(255) not null,  
    contact_number varchar(10) not null,  
    first_login_required boolean not null default true  
);  
  
create table if not exists students (  
    id bigint primary key auto_increment,  
    name varchar(120) not null,  
    enrollment_number varchar(40) not null unique,  
    password_hash varchar(255) not null,  
    program varchar(160) not null,  
    email varchar(180) not null unique,  
    contact varchar(10) not null,  
    first_login_required boolean not null default true  
);  
  
create table if not exists teams (  
    team_id bigint primary key auto_increment,  
    team_name varchar(40) not null unique,  
    task_name varchar(180) not null,  
    teacher_id bigint not null,  
    deadline date not null,  
    status varchar(30) not null,  
    screenshot_path varchar(255),  
    reminder_sent boolean default false,  
  
    teacher_feedback text,  
    review_status varchar(30) default 'PENDING_REVIEW',  
    reviewed_at timestamp null,  
  
    constraint fk_teams_teacher  
    foreign key (teacher_id)  
    references teachers(id)  
);  
  
create table if not exists team_members (  
    id bigint primary key auto_increment,  
    team_id bigint not null,  
    student_id bigint not null,  
    is_leader boolean not null default false,  
  
    constraint fk_team_members_team  
    foreign key (team_id)  
    references teams(team_id)  
    on delete cascade,  
  
    constraint fk_team_members_student  
    foreign key (student_id)  
    references students(id),
```

```

        constraint uq_team_member unique (team_id, student_id),
        constraint uq_student unique (student_id)
    );

create table if not exists activity_logs (
    id bigint primary key auto_increment,
    team_id bigint not null,
    actor_role varchar(30) not null,
    actor_display_name varchar(180) not null,
    action_type varchar(60) not null,
    description varchar(255) not null,
    created_at timestamp not null default current_timestamp,

    constraint fk_activity_logs_team
    foreign key (team_id)
    references teams(team_id)
    on delete cascade
);

create table if not exists password_reset_token (
    id bigint auto_increment primary key,
    email varchar(255) not null,
    token varchar(255) not null unique,
    expiry_time datetime not null
);

create table if not exists reminder_settings (
    id bigint auto_increment primary key,
    reminder_days int
);

create event if not exists delete_expired_tokens
on schedule every 10 minute
do
delete from password_reset_token
where expiry_time < now();

insert into super_admins (
    name,
    email,
    password_hash,
    contact_number
)
select
    'TaskSphere Admin',
    'admin@tasksphere.com',

    '$2a$10$4sR4iK0iM0RKZ77N8D66vOhKcbGaVbL50DJBsSTXobHxPPH/FkN6O',
    '90000000000'
    where not exists (
        select 1 from super_admins
    );

INSERT INTO reminder_settings (id, reminder_days)
SELECT 1, 2
WHERE NOT EXISTS (SELECT 1 FROM reminder_settings);

```

A.2 application.properties — Configuration

```
spring.application.name=TaskSphere
server.port=9090
server.servlet.session.timeout=30m
server.servlet.session.cookie.http-only=true

spring.datasource.url=jdbc:mysql://localhost:3306/tasksphere_db
    ?createDatabaseIfNotExist=true&useSSL=false
    &allowPublicKeyRetrieval=true&serverTimezone=Asia/Kolkata
spring.datasource.username=root
spring.datasource.password=<your_password>
spring.datasource.driver-class-name=com.mysql.cj.jdbc.Driver

spring.sql.init.mode=always
spring.sql.init.schema-locations=classpath:schema.sql
spring.sql.init.data-locations=classpath:data.sql

spring.thymeleaf.cache=false

spring.servlet.multipart.max-file-size=10MB
spring.servlet.multipart.max-request-size=10MB

spring.mail.host=smtp.gmail.com
spring.mail.port=587
spring.mail.username=<gmail_address>
spring.mail.password=<app_password>
spring.mail.properties.mail.smtp.auth=true
spring.mail.properties.mail.smtp.starttls.enable=true

tasksphere.upload-dir=uploads
```

APPENDIX B: INSTALLATION GUIDE

B.1 Prerequisites

- JDK 21 installed; JAVA_HOME environment variable set correctly.
- MySQL 8.x server running on localhost:3306.
- Apache Maven 3.x installed and available on the system PATH.
- A Gmail account with two-factor authentication enabled and an App Password generated for use as the SMTP credential.
- Git installed (optional, for cloning the repository).

B.2 Setup and Run

17. Clone or extract the project: `git clone <repository-url>` OR `unzip TaskSphere.zip`
18. Navigate to the project directory: `cd TaskSphere`
19. Edit `src/main/resources/application.properties`: set `spring.datasource.username`, `spring.datasource.password`, `spring.mail.username`, and `spring.mail.password`.
20. Build the application: `mvn clean package -DskipTests`
21. Run the application: `java -jar target/tasksphere-0.0.1-SNAPSHOT.jar`
22. On startup, Spring executes `schema.sql` (creates all tables) and `data.sql` (inserts the default Super Admin and `reminder_settings` seed row).
23. Open a browser and navigate to: `http://localhost:9090/login`

B.3 Default Super Admin Login

```
Role:      SUPER_ADMIN
Email:     admin@tasksphere.com    (as seeded in data.sql)
Password:  Admin@123              (BCrypt hash stored; change after first
login)
```

B.4 MySQL Event Scheduler

The `delete_expired_tokens` MySQL EVENT (defined in `schema.sql`) requires the MySQL event scheduler to be enabled. Verify with:

```
SHOW VARIABLES LIKE "event_scheduler";    -- should return "ON"
```

If it returns "OFF", enable it with:

```
SET GLOBAL event_scheduler = ON;
```

Or add `event_scheduler=ON` to your MySQL server's `my.cnf` / `my.ini` configuration file for persistence across restarts.